ETC

Indice

1 Nozione di Base ETC 3
1.1 Insiemi 3
1.2 Logica Booleana 8
1.3 Dimostrazioni 10
1.4 Grafi 11
1.5 Alfabeto 12
2 Automi Finiti Deterministici 17
2.1 Usi DFA 17
2.2 Parti DFA 17
2.3 Definizione Formale DFA 18
2.4 Relazione fra DFA e Linguaggi 18
2.5 Dal DFA al Linguaggio Riconosciuto 22
2.6 Liguaggi Regolari 27
2.7 Calcolo Correttezza Automa 29
2.8 Esempio Tecnica Progettazione Diretta DFA senza Calcolo Correttezza 30
2.9 Chiusura Linguaggi Regolari e Dimostrazione Correttezza DFA 31
2.10 Teorema 1.25 Chiusura Classe REG Unione 32
2.11 Teorema 1.25 Bis Chiusura Classe REG Intersezione 34
2.12 Teorema (4.5 HUM) Chiusura Classe REG Complemento 41
2.13 Teorema 1.26 Chiusura Classe REG Concatenazione 41
2.14 Organizzazione Riposta DFA con Teorema Chiusura 41
3 Automi Finiti Non Deterministici 42
3.1 Linguaggio Accettato o Riconosciuto NFA 44
3.2 Semplici NFA 44
3.3 Esempio NFA 44
3.4 Definizione Formale NFA 47
3.5 Epsilon-Chiusura (HUM 2.5.3-2.5.4) 47
3.6 Funzione di Transizione Estesa NFA 49

3.7 Teorema 1.45 Chiusura Classe REG Unione NFA 51
3.8 Teorema 1.47 Chiusura Classe REG Concatenazione NFA 52
3.9 Teorema 1.49 Chiusura Classe REG Chiusura di Kleene 53
3.10 Chiusura Classe REG Reverse 54
3.11 Chiusura Classe REG Prefissi e Suffissi 56
4 Ogni NFA è un DFA e Viceversa 58
4.1 Ogni DFA è un NFA 59
4.2 Teorema 1.39 NFA e DFA equivalenti 60
5 Espressioni Regolari 62
5.1 Definizione Formale Espressioni Regolari 62
5.2 Precedenza Operatori 62
5.3 Definizione Ricorsiva Linguaggi Rappresentati REG 62
5.4 Proprietà Algebriche RE 63
6 Pumping Lemma 64
6.1 Tecnica Per Provare la Non Regolarità 64
7 Macchine di Turing 65
7.1 Descrizione Informale 65
7.2 Definizione Formale MdT 65
7.3 Funzione di Transizione MdT 65
7.4 Diagramma di Stato 65
7.5 Computazione di una Mdt Informale 66
7.6 Configurazione di una MdT 66
7.7 Passo di Computazione 67
7.8 Computazione 67
7.9 Parola Accetta da una MdT 68
7.10 Parola Rifiutata da una MdT 68
7.11 Linguaggio Riconosciuto da una MdT 69
7.12 MdT Decisore 69
7.13 Linguaggio Deciso 69
7.14 Linguaggio Riconoscibile 70
7.15 Linguaggio Decidibile 70
7.16 Funzioni Calcolabili 70

1 Nozione di Base ETC

1.1 Insiemi

Gli insiemi sono collezioni non ordinata di oggetti o elementi distinti

- Ordine e ridondanza non contano
- **Insieme universale U** è l'insieme di tutti gli elementi pertinenti al contesto

1.1.1 Metodi di rappresentazione

- Insieme finito può essere descritto dalla lista dei suoi elementi separati da virgola tra {}
- Insieme infinito può essere descritto mediante la notazione "..."
  - $N = \{0, 1, 2, \ldots, n, \ldots\}$
- Altro modo per definire un insieme è di specificare la proprietà caratteristica dei suoi elementi
  - $A = \{w \mid w \text{ ha la proprietà } P\}$

1.1.1.1 Esempio di rappresentazione

- L' insieme dei numeri interi non negativi pari è
  - $\{0, 2, 4, \ldots, 2n, \ldots\} = \{2n \mid n \in N, n \geq 0\}$
- L' insieme dei numeri interi positivi pari è
  - $\{2, 4, \ldots, 2n, \ldots\} = \{2n \mid n \in N, n \geq 1\}$
- L' insieme dei numeri interi non negativi dispari è
  - $\{1, 3, 5, 7, \ldots, 2n + 1, \ldots\} = \{2n + 1 \mid n \in N, n \geq 0\}$

1.1.2 Definizione Ricorsiva (o Induttiva)

Una definizione ricorsiva di un insieme è composta da:

- Passo base: Definisce uno o più elementi elementari dell'insieme
- Passo ricorsivo: Definisce regole per formare nuovi elementi dell'insieme a partire da quelli già definiti

1.1.2.1 Esempio Definizione Ricorsiva

Sia A l'insieme definito ricorsivamente nel modo seguente

- Passo base: i1 = 1 ∈ A.
- Passo ricorsivo: Se ik ∈ A allora ik+1 = ik + 2 ∈ A.

1.1.3 Cardinalità

La cardinalità |S| di un insieme finito S è il numero di elementi in S

- Insieme vuoto ∅ ha cardinalità 0

1.1.3.1 Lemma

Se S e T sono finiti allora $|S \cup T| = |S| + |T| - |S \cap T|$

- Se S e T sono disgiunti allora $|S \cup T| = |S| + |T|$

1.1.4 Sottoinsieme

Siano S e T insiemi, diciamo che S ⊆ T se w ∈ S implica w ∈ T

- Ogni elemento di S è anche un elemento di T

1.1.4.1 Esempio Sottoinsieme

- S = {ab, ba} e T = {ab, ba, aaa} allora S ⊆ T ma $T \nsubseteq S$
- S = {ba, ab} e T = {aa, ba} allora $S \nsubseteq T$ e $T \nsubseteq S$

1.1.5 Insieme Uguali

Due insiemi T e S sono uguali se S ⊆ T e T ⊆ S

1.1.5.1 Esempio Insiemi Uguali

- Siano S = {ab, ba} e T = {ba, ab}, allora S ⊆ T e T ⊆ S; quindi S = T

1.1.6 Unione

Dati due insiemi S e T, la loro unione è l'insieme

- $S \cup T = \{ w \mid w \in S \text{ oppure } w \in T\}$

1.1.6.1 Esempio Unione

- $S = \{a, ba\}$ e $T = \emptyset$, allora $S \cup T = S$
- $S = \{a, ba\}$ e $T = \{\epsilon\}$ allora $S \cup T = \{\epsilon, a, ba\}$

1.1.7 Intersezione

Dati due insiemi S e T, la loro intersezione è l'insieme

- $S \cap T = \{w \mid w \in S \text{ e } w \in T\}$

Due insiemi si dicono **disgiunti** se $S \cap T = \emptyset$

1.1.8 Differenza

Dati due insiemi S e T, la loro differenza è

- $S - T = \{ w \mid w \in S \text{ e } w \notin T\}$

1.1.8.1 Esempio Differenza

- Sia $S = \{a, b, bb, bbb\}$ e $T = \{a, bb, bab\}$ allora $S - T = \{b, bbb\}$.
- Sia $S = \{ab, ba\}$ e $T = \{ab, ba\}$ allora $S - T = \emptyset$

1.1.9 Complemento

Dato un insieme universale U, il completo di un insieme $S \subseteq U$ è

- $\overline{S} = \{w \mid w \in U, w \notin S\} = U - S$
- $\overline{S}$ è l'insieme di tutti gli elementi di U che non sono in S

1.1.10 Sequenze

Una sequenza di oggetti è una lista di oggetti in un qualche ordine

- Denotata scrivendo la lista fra ()
- Ordine e ridondanza sono importanti

Una **k-upla** è una sequenza di k elementi

1.1.11 Prodotto Cartesiano

Dati due insiemi A e B, il prodotto cartesiano A x B è l'insieme di coppie

- A x B = {(x, y) | $x \in A$, $y \in B$}
- A x B $\neq$ B x A
- $\emptyset$ x B = $\emptyset$
- Se A e B sono insiemi finiti, |A x B| = |A||B|

1.1.11.1 Esempio Prodotto Cartesiano

- Siano A = {a, ba, bb} e B = {ba}, allora
- A x B = {(a, ba),(ba, ba),(bb, ba)}
- B x A = {(ba, a),(ba, ba),(ba, bb)

1.1.11.2 Prodotto Cartesiano di due o più insiemi

- A1 x . . . x Ak = {(x1, . . . , xk) | xi ∈ Ai, 1 ≤ i ≤ k}

1.1.12 Funzioni

Dati due insiemi non vuoti X e Y, una funzione f : X -> Y è una relazione che associa ad ogni elemento di x in X uno e uno solo elemento y = f(x) in Y

- X Dominio
- Y Codominio
- Occorre definire anche la relazione che associa ogni elemento del dominio a un elemento del codominio
- Un **predicato** o **proprietà** è una funzione il cui range è $\{true, false\}$
- Una **proprietà** il cui dominio è un insieme di k-tuple è detto **relazione**, di due tuple è detta **relazione binaria**
  - Si dice **relazione di equivalenza** se è riflessiva (se per ogni x, xRx), transitiva (se per ogni x e y e z, xRy e yRz implica xRz) e simmetrica (se per ogni x e y, xRy implica yRx)

1.1.12.1 Classificazioni Funzioni

- Una funzione $f : X \to Y$ è **iniettiva** se

$$\forall x, x' \in X, x \neq x' \Rightarrow f(x) \neq f(x')$$

- Una funzione $f : X \to Y$ è **suriettiva** se

$$\forall y \in Y, \exists x \in X : y = f(x)$$

- Una funzione $f : X \to Y$ è **biettiva** di X su Y se f è iniettiva e suriettiva

1.1.13 Insieme Potenza

Per ogni insieme S, l'insieme potenza (o insieme delle parti) $P(S)$ è l'insieme di tutti i possibili sottoinsiemi di S

$$P(S) = \{A \mid A \subseteq S\}$$

1.1.13.1 Lemma

- Se S è un insieme finito allora

$$|P(S)| = 2^{|S|}$$

1.2 Logica Booleana

Un'espressione booleana Φ sulle variabili $x_1, ..., x_n$ è soddisfatta da un assegnamento che rende vera Φ

- Vero = 1
- Falso = 0

1.2.1 Negazione o Not

Rappresenta il valore opposto e si usa il simbolo $\neg$

1.2.2 Congiunzione o And

Vera se entrambi i valori sono veri e si usa il simbolo $\wedge$

1.2.3 Disgiunzione o Or

Vera se almeno uno dei due valori è vero e si usa il simbolo $\vee$

1.2.4 Or Esclusivo

Vero se esattamente uno dei due operandi è 1 e usa il simbolo $\oplus$

1.2.5 Uguaglianza

Vera se entrambi gli operandi hanno lo stesso valore e si usa il simbolo $\leftrightarrow$

1.2.6 Implicazione

Falso se il suo primo operando ha valore vero e il secondo falso, vero in tutti gli altri casi $\rightarrow$

- $0 \rightarrow 0 = 1$
- $0 \rightarrow 1 = 1$
- $1 \rightarrow 0 = 0$
- $1 \rightarrow 1 = 1$

1.2.7 Relazioni fra Operazioni

- $P \vee Q = \neg(\neg P \wedge \neg Q)$
- $P \rightarrow Q = \neg P \vee Q$
- $P \leftrightarrow Q = (P \rightarrow Q) \wedge (Q \rightarrow P)$
- $P \oplus Q = \neg(P \leftrightarrow Q)$

1.2.8 Legge Distributiva

- $P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$
- $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$

1.3 Dimostrazioni

Ragionamento convincente per dire che un enunciato è vero

- Se un enunciato è falso si usa il controesempio
- Dimostrazione per **assurdo**: Assumiamo che il teorema sia falso per poi dimostrare che questa ipotesi porta ad una contraddizione o assurdo
- Dimostrazione **costruttiva**: Mostrare come costruire l'oggetto
- Dimostrazione per **induzione**: Permette di dimostrare che tutti gli elementi di un insieme infinito godono di una data proprietà
  - Base induttiva: Dimostra che $P(1)$ vero
  - Passo induttivo: Se $P(1)$ vero (**ipotesi induttiva**), allora per ogni $i \geq 1$ anche $P(i+1)$ vero
- **Lemma**: Enunciato interessante che serve alla dimostrazione di un enunciato più interessante

1.4 Grafi

Un grafo non orientato è un insieme di punti e di linee che connettono alcuni dei punti

- Punti chiamati **nodi**
- Linee chiamate **archi**
- Numero di archi rappresenta il grado del grafo

1.4.1 Sottografo

Diciamo che il grafo G è un sottografo di H se i nodi di G sono un sottoinsieme dei nodi di H e gli archi di G sono anche archi di H sui nodi corrispondenti

1.4.2 Cammino

Un cammino in un grafo è una sequenza di nodi collegati da archi

- **Cammino semplice**: Un cammino che non contiene nodi ripetuti
- **Grafo connesso**: Se esiste un cammino per ogni coppia di nodi
- **Ciclo**: Un cammino che inizia e termina nello stesso nodo
- **Ciclo semplice**: Contiene almeno tre nodi e ripete solo il primo e ultimo nodo

1.4.3 Albero

Un grafo connesso senza cicli semplici

- In un albero con n nodi ci sono sempre esattamente $n - 1$ archi
- Presenta una **radice** e i nodi di grado 1 ad eccezione della radice sono detti **foglie**

1.4.4 Grafo Orientato o Diretto

Presenta delle frecce che indicano la direzione

- Il numero di frecce che escono da un nodo è il **grado uscente**, mentre il numero di frecce che entrano è detto **grado entrante**
- Archi rappresentati da una coppia ordinata, dove l'ordine indica la direzione
- Un grafo orientato G è (V, E), con V insieme dei nodi ed E è l'insieme degli archi
- Un cammino in cui tutte le frecce puntano nella stessa direzione è detto **cammino orientato** ed è **fortemente connesso**

1.5 Alfabeto

Un insieme finito di elementi.
Sia $\Sigma = \{a_1, ..., a_k\}$ un alfabeto con k simboli, la cardinalità di $|\Sigma|$ è k

1.5.1 Stringa

Su un alfabeto è una sequenza di simboli dell'alfabeto

- La stringa vuota ε non contiene nessun simbolo
- Lunghezza della stringa s è denotata dal numero di simboli |s|

1.5.1.1 Definizione Ricorsiva di Stringa

- Passo base: La stringa vuota ε è una stringa
- Passo ricorsivo: Se w è una stringa e $x \in \Sigma$ è un simbolo dell'alfabeto, allora wx è una stringa

1.5.2 Σ^*

Dato un alfabeto Σ, denotiamo con Σ^* l'insieme di tutte le possibili stringhe su Σ

- Σ e Σ^* sono insiemi diversi

1.5.2.1 Definizione Ricorsiva di Σ^*

- Passo base: $\varepsilon \in \Sigma^*$
- Passo ricorsivo: Se $w \in \Sigma^*$ e $x \in \Sigma$, allora $wx \in \Sigma^*$

1.5.2.2 Definizione Ricorsiva di Lunghezza di una Stringa

- Passo base: $|\varepsilon| = 0$
- Passo ricorsivo: Se w è una stringa e $x \in \Sigma$, allora $|wx| = |w| + 1$

1.5.3 Stringhe Eguali

Due stringhe

$$x = a_1 a_2 ... a_h$$

e

$$y = b_1 b_2 ... b_k$$

con a_i e $b_j \in \Sigma$, $1 \leq i \leq h$, $1 \leq j \leq k$
si dicono eguali se $h = k$ e $a_i = b_i$; per $i = 1, ..., h$

- Caratteri letti da destra e sinistra sono uguali

1.5.4 Concatenazione di Stringhe

Date le stringhe $x = a_1a_2...a_h$, $y = b_1b_2...b_k$, con $a_i, b_j \in \Sigma$, $1 \leq i \leq h$, $1 \leq j \leq k$, la concatenazione è definita come

$$x \cdot y = a_1a_2...a_hb_1b_2...b_k$$

Inoltre

$$x\varepsilon = \varepsilon x = x$$

- Non è commutativa, Siano x e y stringhe $xy \neq yx$
- La concatenazione è associativa, Siano x,y,z stringhe $(xy)z = x(yz)$
- Lunghezza di $|xy| = |x| + |y|$

1.5.4.1 Definizione Ricorsiva di Concatenazione

- Passo base: Se $w \in \Sigma^*$ allora $w\varepsilon = w \in \Sigma^*$
- Passo ricorsivo: Se $w_1 \in \Sigma^*, w_2 \in \Sigma^*, w_2 \neq \varepsilon, \exists w_2 \in \Sigma^*, x \in \Sigma$ tali che $w_2 = w_2'x$ allora $w_1 \cdot w_2 = w_1 \cdot (w_2'x) = (w_1 \cdot w_2')x \in \Sigma^*$

1.5.5 Potenza di una Stringa

Sia $m \geq 0$ un intero non negativo

- Potenza con esponente $m = 0$ di una stringa è la stringa vuota ε
- Per $m > 0$, la potenza di una stringa w è la concatenazione di w con se stessa $m - 1$ volte

1.5.5.1 Definizione Ricorsiva di Potenza

- Passo base: $w^0 = \varepsilon$
- Passo ricorsivo: $w^m = w^{m-1}w$, per $m > 0$

1.5.6 Sottostringa

w è una sottostringa di s se esistono stringhe x e y tali che $s = xwy$

1.5.6.1 Esempio Sottostringa

- 567 è sottostringa di 34567
- 42 non è sottostringa di 472

1.5.7 Inversa di una Stringa

L'inversa w^R di una stringa w è la stringa ottenuta scrivendo i caratteri da destra verso sinistra

$$\varepsilon^R = \varepsilon$$

e se

$$w = a_1 a_2 ... a_n$$

con a_j simboli

$$w^R = a_n a_{n-1} ... a_1$$

1.5.7.1 Definizione Ricorsiva dell'Inversa di una Stringa

- Passo base: $\varepsilon^R = \varepsilon$
- Passo ricorsivo: Per ogni $x \in \Sigma^*$ e $\sigma \in \Sigma$, $(x\sigma)^R = \sigma x^R$

1.5.8 Precedenze

- $(ab)^2 = abab \neq abb = ab^2$
- $(ab)^R = ba \neq ab^R = ab$

1.5.9 Linguaggi

Un linguaggio è un insieme di stringhe su un alfabeto $L \subseteq \Sigma^*$

- $\overline{\mathrm{L}} = \Sigma^* - L = \{w \in \Sigma^* \mid w \notin L\}$
- Non sono sempre finiti
- Cardinalità è il numero delle sue stringhe $|L| = |\{aba, aab\}| = 2$
- $|\emptyset| = 0$
- $\varepsilon \notin \emptyset$
- $\emptyset \neq \{\varepsilon\}$

1.5.9.1 Esempio Linguaggi 1

- Sia $\Sigma = \{a, b\}$
- $L_1 = \{aa, aaa\}$

1.5.9.2 Esempio Linguaggi 2

- Alfabeto {a, b}
- Linguaggio $L = \{w \in \{a, b\}^* | \text{la prima lettera di w è b}\}$
- L: insieme delle stringhe su {a, b} che non iniziano con b.
- NON insieme stringhe che iniziano con a (es. stringa vuota ϵ ∈ L)

1.5.10 Prodotto di Linguaggi

Dati due linguaggi S e T sull'alfabeto Σ, il prodotto è l'insieme di tutte le stringhe che sono concatenazione di una stringa S e una stringa T

$$ST = S \cdot T = \{uv \in \Sigma^* \mid u \in S, v \in T\}$$

- Diversa dal prodotto cartesiano in cui consideriamo le coppie
  - $S \times T = \{(a, \epsilon), (a, ba), (ba, \epsilon), (ba, ba), (bb, \epsilon)\}, ST \neq S \times T$
- Nel prodotto di linguaggi il risultato è un insieme di stringhe (concatenate), mentre nel prodotto cartesiano è un insieme di tuple

1.5.10.1 Esempio Prodotto di Linguaggi

- Se $S = \{a, aa\}$ e $T = \{\epsilon, a, ba\}$, allora
- $ST = \{a, aa, aba, aaa, aaba\}, TS = \{a, aa, aaa, baa, baaa\}$

1.5.11 Potenza di un Linguaggio

Sia L un linguaggio sull'alfabeto Σ

$$L^0 = \{\varepsilon\}$$

$$L^k = L^{k-1}L, k \geq 1$$

1.5.12 Casi Particolari Operazioni sui Linguaggi

- $\emptyset^0 = \{\varepsilon\}$
- $L \cdot \emptyset = \emptyset \cdot L = \emptyset$
- $L \cdot \{\varepsilon\} = \{\varepsilon\} \cdot L = L$

1.5.13 Chiusura di Kleene

La chiusura di Kleene di un linguaggio L è

$$L^* = \bigcup_{n \geq 0} (L^n)$$

- L^* è il linguaggio ottenuto concatenando un numero qualsiasi di stringhe L
- $L^* = \{w_1 w_2 ... w_k \mid k \geq 0, w_i \in L, 1 \leq i \leq k\}$
- Con $k = 0$, $w_1 w_2 ... w_k = \varepsilon$

1.5.13.1 Esempio Chiusura di Kleene

- Se $L = \{a, b\}$, allora $L^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, ...\}$
- Se $L = \emptyset$, allora $L^* = \{\varepsilon\}$
- Se $L = \{\varepsilon\}$, allora $L^* = \{\varepsilon\}$
- $(L^*)^* = L^*$

1.5.14 Chiusura Positiva

La chiusura positiva si distingue dalla chiusura di Kleene perchè nell'unione non compare la potenza L^0

$$L^+ = \bigcup_{n > 0} L^n = \{w_1 w_2 ... w_k \mid k > 0, w_i \in L, 1 \leq i \leq k\}$$

1.5.14.1 Esempio Chiusura Positiva

- Se $L = \{a\}$, allora
- $L^+ = \{a, aa, aaa...\} = \{a^n \mid n > 0, n \in \mathbb{N}\}$
- $L^+ = \bigcup_{n>0} (L^n) = \emptyset = L$
- $L^+ = \bigcup_{n>0} (L^n) = \{\varepsilon\} = L$

2 Automi Finiti Deterministici

Meccanismo di computazione con poca memoria

- Descrivono comportamenti
- Ricevono in input delle stringhe e risponderanno accettando o rifiutando l'input
- Numero di stati finito
- Unico Stato iniziale
- **Deterministico**: Se e solo se ha esattamente 1 transizione da ogni stato per ogni simbolo

2.1 Usi DFA

- Prima fase del processo di compilazione
  - Legge il codice sorgente e controlla la "correttezza" di quanto scritto: raggruppa i caratteri, interagendo con una tavola di simboli e automi
  - Espande le macro, prepara messaggi di errore, elimina spazi, linee, commenti…
  - Produce in output un flusso di token (ad esempio: numero, operatore) associato ai lessemi (ad esempio: 17, +, :)
- Automi per il pattern matching sia per la bioinformatica che per la sicurezza

2.2 Parti DFA

- Presenta degli stati
  - Es. $\{q_0, q_1, q_2, q_3, q_4\}$
- Stato iniziale è lo stato da cui inizia l'input, è unico
- Stati accettati che portano ad accettare l'input
- Stati non accettati che portano a rifiutare l'input
- Presenta un alfabeto per l'input
- Stato pozzo o Trappola, non è mai accettante, definisce uno stato non accettante per tutti i simboli dell'alfabeto e dal quale non si può uscire non avendo archi uscenti

2.3 Definizione Formale DFA

Un DFA M è una 5-tupla $(Q, \Sigma, \delta, q_0, F)$

- $\boldsymbol{Q}$:Insieme degli stati
- $\boldsymbol{\Sigma}$: Alfabeto delle stringhe in input
- $\boldsymbol{\delta}$: Funzione di transizione tra gli stati $\delta : Q \times \Sigma \to Q$
  - Associa ad una coppia (stato, carattere) un solo stato
  - Può essere definito come: diagramma di stato, tabella di transizione o scrittura esplicita della funzione
  - Esattamente una transizione da ogni stato per ogni simbolo dell'alfabeto (**deterministico**)
- $\boldsymbol{q_0}$: Stato iniziale
- $\boldsymbol{F}$: Insieme degli stati accettati, $F \subseteq Q$
  - Può essere vuoto (accetta solo $\emptyset$), coincidere con l'insieme degli stati Q (accetta tutto Σ^*), o essere un suo sottoinsieme

2.4 Relazione fra DFA e Linguaggi

Dato un automa M, con $L(M)$ indichiamo il linguaggio che l'automa riconosce, quindi il linguaggio L è riconosciuto da un automa M se $L(M) = L$

Ogni DFA accetta un insieme di stringhe, quindi un linguaggio L, rifiutando quelle che non sono in L

- Per ogni stringa in Σ^* l'automa M accetta o non accetta
- Il linguaggio che un DFA riconosce è l'insieme delle stringhe di input per le quali il DFA accetta, ed è **unico**

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito deterministico, il liguaggio accettato o riconosciuto da A è

$$L(A) = \left\{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \right\}$$

- **Cammino Accettante**: Cammino che ha come ultimo nodo un nodo accettante
  - Se esiste un cammino accettante, tale cammino è unico (come tutti i cammini)
- Non è possibile costruire gli automi per ogni elemento di Σ^*

- Dato un linguaggio L possiamo avere più automi
- Dato un automa M riconosce un solo linguaggio unico
- Dato un linguaggio L, costruire se possibile un automa M che lo riconosce equivale a dire $L = L(M)$

2.4.1 Descrizione Formale delle Stringhe che l'Automa Accetta

Sia w una stringa $w = a_1 a_2 ... a_n, a_i \in \Sigma$
L'automa M accetta w se e solo se

$$\exists (r_0, r_1 ... r_n), r_i \in Q, r_0 = q_0, r_n \in F$$

e

$$\forall i = 0, 1, ..., n-1, \delta(r, a_i) = r_{i+1}$$

- Quindi la macchina inzia da una condizione iniziale che appartiene alla sequenza di stati
- La macchina passa da uno stato ad un altro in base ad una funzione di transizione
- La macchina accetta il suo input se termina la lettura in uno stato accettante

$$L(M) = \{w \mid M \text{ accetta } w\}$$

2.4.2 Descrizione Ricorsiva Formale delle Stringhe che l'Automa Accetta

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito deterministico
La funzione di transizione estesa $\hat{\delta} : Q \times \Sigma^* \to Q$ è definita ricorsivamente come segue:

- Passo base:

$$\forall q \in Q, \hat{\delta}(q, \varepsilon) = q$$

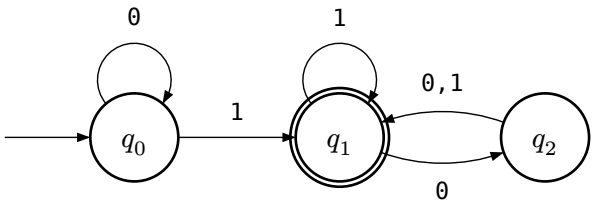
Non è la transizione, ma $\hat{\delta}$ è uno stato

- Passo Ricorsivo:

$$\forall q \in Q, w \in \Sigma^*, a \in \Sigma,$$

$$\hat{\delta}(q, wa) = \delta\Big(\hat{\delta}(q, w), a\Big)$$

2.4.2.1 Esempio Descrizione Ricorsiva Formale delle Stringhe che l'Automa Accetta



Sia $w = 0100$

$$\hat{\delta}(q_0, 0100) = \delta\Big(\hat{\delta}(q_0, 010), 0\Big) =$$

$$\delta\Big(\delta\Big(\hat{\delta}(q_0, 01), 0\Big), 0\Big) =$$

$$\delta\Big(\delta\Big(\delta\Big(\hat{\delta}(q_0, 0), 1\Big), 0\Big), 0\Big) =$$

$$\delta\Big(\delta\Big(\delta\Big(\delta\Big(\hat{\delta}(q_0, \varepsilon), 0\Big), 1\Big), 0\Big), 0\Big) =$$

Ma $\hat{\delta}(q_0, \varepsilon)$ è il passo base quindi:

$$\delta(\delta(\delta(\delta(q_0, 0), 1), 0), 0)$$

Prova della correttezza:

$$\hat{\delta}(q_0, 0100) = \delta(\delta(\delta(\delta(q_0, 0), 1), 0), 0) =$$

$$\delta(\delta(\delta(q_0, 1), 0), 0) =$$

$$\delta(\delta(q_1, 0), 0) =$$

$$\delta(q_2, 0) = q_1$$

2.4.3 DFA e Linguaggi con Cicli e senza Cicli

- L'assenza di cicli in un automa finito M lungo cammini di accettazione implica che $L(M)$ è finito
- La presenza di un ciclo non implica che il linguaggio sia infinito, ma occorre che sia su un cammino accettante

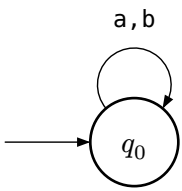
2.4.4 Automa Insieme Vuoto

- $L(M) = \{\}$

$$(Q, \Sigma, \delta, q_0, F) \text{ dove } Q = \{q_0\}, \Sigma = \{a, b\}, F = \emptyset$$

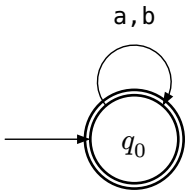
$$\delta(q_0, a) = \delta(q_0, b) = q_0$$

- Non è unico

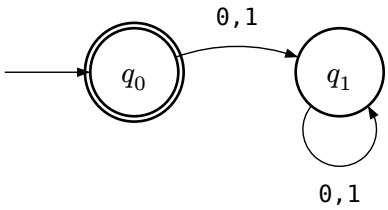


2.4.5 Automa Σ^*

- $L(M) = \Sigma^*$



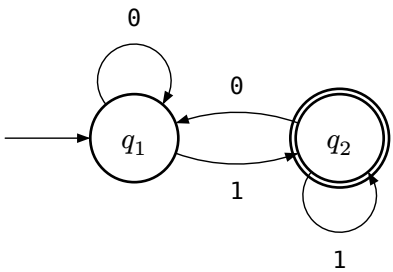
2.4.6 Automa Solo Parola Vuota



2.5 Dal DFA al Linguaggio Riconosciuto

Complicato ma sempre possibile grazie ad algoritmi appositi

2.5.1 Esempio 1.7



Forniamo la descrizione formale:

$$M_2 = (\{q_1, q_2\}, \{0, 1\}, \delta, q_1, \{q_2\})$$

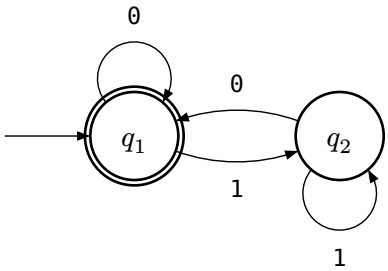
$$\delta(q_1, 0) = q_1$$
$$\delta(q_2, 0) = q_1$$
$$\delta(q_1, 1) = q_2$$
$$\delta(q_2, 1) = q_2$$

Non è presente la parola vuota

$$L(M_2) = \{w \text{ su } \{0,1\} \mid w \text{ termina con } 1\}$$

$$L(M_2) = \{w \in \{0,1\}^* \mid w = w'1, w' \in \{0,1\}^*\} = \{0,1\}^*\{1\}$$

2.5.2 Esempio 1.9



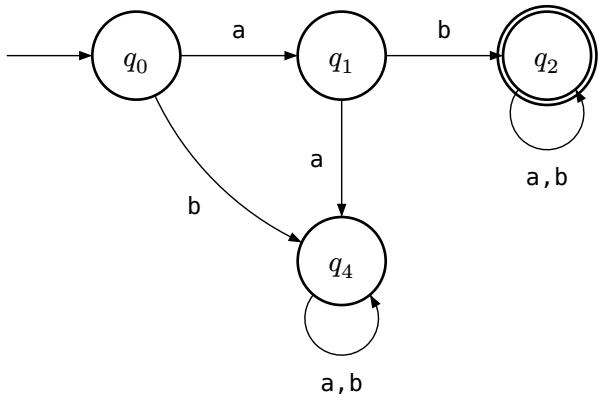
$$M_3 = (\{q_1, q_2\}, \{0,1\}, \delta, q_1, \{q_1\})$$

Parola vuota accettata

$$L(M_3) = \{w \text{ su } \{0,1\} \mid w = \varepsilon \text{ oppure termina con } 0\}$$

$$L(M_3) = \{w \in \{0,1\}^* \mid w = \varepsilon \text{ oppure } w = w'0, w' \in \{0,1\}^*\} = \{0,1\}^*\{0\} \cup \{\varepsilon\}$$

2.5.3 Esempio Slide 1

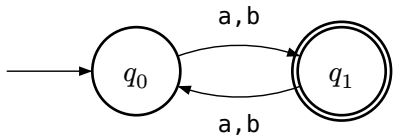


$$L(M) = \{s \mid s \text{ inizia con } ab\} =$$

$$L(M) = \{abw \mid w \in \{a,b\}^*\} =$$

$$L(M) = \{ab\}\{a,b\}^*$$

2.5.4 Esempio Slide 2

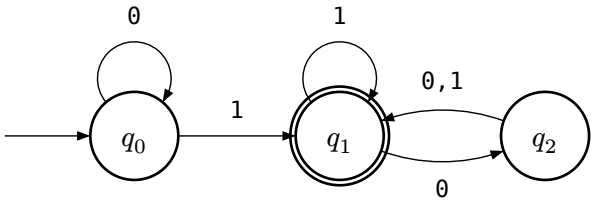


$$L(M) = \{s \mid \text{lunghezza di } s \text{ è dispari}\} =$$

$$L(M) = \{aa, ab, ba, bb\}^*\{a,b\} =$$

$$L(M) = (\{a,b\}\{a,b\})^*\{a,b\}$$

2.5.5 Esempio 1.6



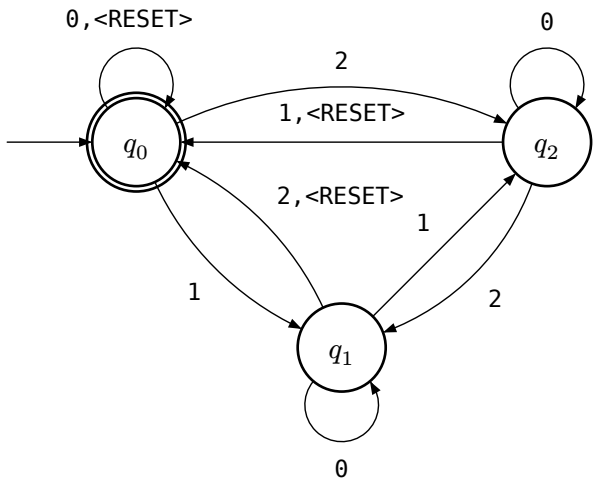
$L(M) = \{w$ in $\{0,1\}^*|$ w termina con 1 oppure
tale che dopo l'ultimo 1 ha un numero pari di $0\} =$
Consideriamo che $w = w'1w''$ dove $w' \in \{0,1\}^*, w'' \in \{00\}^*$

2.5.6 Esempio 1.13

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{\text{RESET}, 0, 1, 2\}$$

- Con RESET torno nello stato iniziale q_0
- Leggere 0 non fa cambiare stato
- Leggere 1 mi fa spostare nello stato successivo
- Riconosce sicuramente solo RESET e solo 0
- Se non consideriamo RESET allora accetta le parole in cui la somma è $0 \bmod 3$
- Quindi il linguaggio accettato dall'automa riguarda le parole in cui la somma è $0 \bmod 3$ e con RESET la somma viene azzerata, la macchine conserva un conto parziale e RESET lo riporta a 0 ed accetta i multipli di 3



2.6 Liguaggi Regolari

Un linguaggio L è definito regolare se e solo se esiste un DFA M che lo riconosce

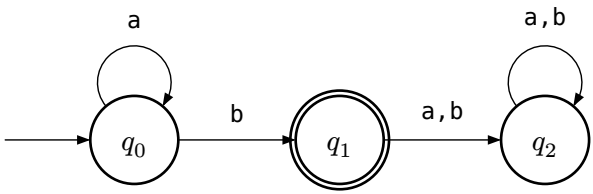
$$L = L(M)$$

- Non tutti i linguaggi sono regolari, ad es. $\{a^n b^n \mid n \geq 0\}$, dato che ogni potenza aggiunge un nuovo stato e l'automa si allunga all'infinito

2.6.1 Esempio Linguaggi Regolari

- Tutti i linguaggi finiti
- $\{a^n b| \ n \geq 0\}$

$$L = \{a^n b| \ n \geq 0\} = \{b, ab, aab, aaab, ...\}$$

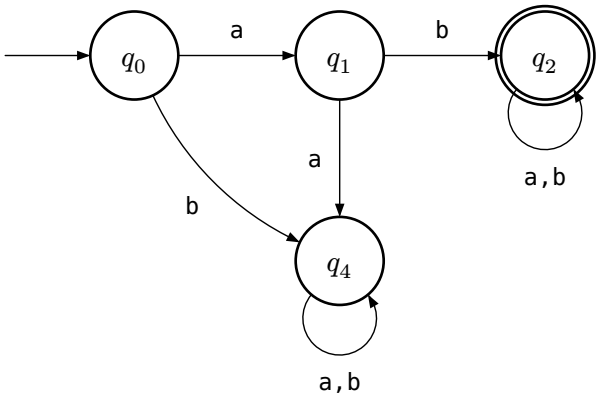


- Tutte le stringhe in $\{a, b\}^*$ con prefisso ab

$$L(M) = \{s \mid s \text{ inizia con } ab\} =$$

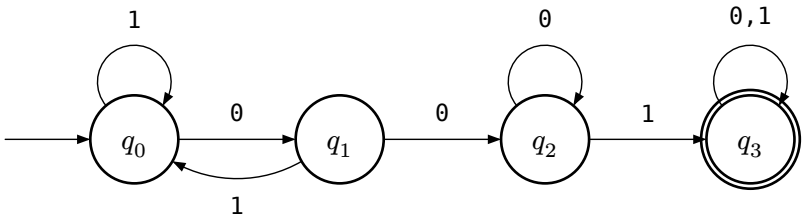
$$L(M) = \{abw \mid w \in \{a, b\}^*\} =$$

$$L(M) = \{ab\}\{a, b\}^*$$



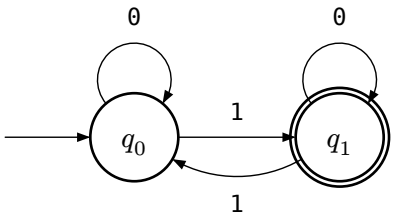
- Tutte le stringhe in $\{0, 1\}^*$ che contengono 001 come fattore

$$L = \{x001y \mid x, y \in \{0, 1\}^*\}$$



Presenza del cappio relativo al numero 1 spezzerebbe il fattore 001

- Tutte le stringhe su $\{0, 1\}$ che hanno un numero dispari di 1



2.7 Calcolo Correttezza Automa

Dovrei dimostrare che se per L ho disegnato un DFA $M = (Q, \Sigma, \delta, q_0, F)$ si ha

$$w \in L \Leftrightarrow \hat{\delta}(q_0, w) \in F$$

Tuttavia la prova per induzione potrebbe non essere facile

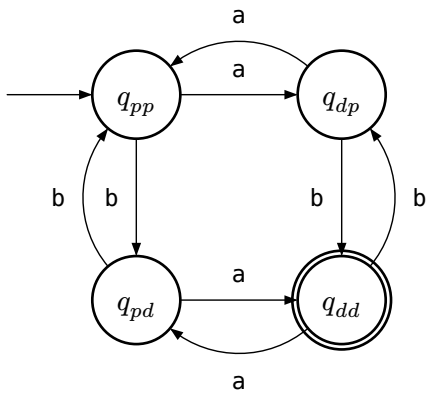
2.8 Esempio Tecnica Progettazione Diretta DFA senza Calcolo Correttezza

- Progettare un DFA che accetta

$$D = \{w \in \{a, b\}^* \mid |w| = 2h, h \geq 0 \text{ e } |w|_a = 2k + 1, k \geq 0\}$$

Quindi devono avere anche un numero dispari di b essendo $\Sigma = \{a, b\}$

$$\forall w \in D \text{ si ha che } |w|_b = 2k + 1, k \geq 0$$



Abbiamo osservato che $\forall w \in D$ si ha che $|w|_b = 2k + 1, k \geq 0$
Quindi sto contando sia a che b dispari, **entrambi** devono essere dispari

$$D_a = \{w \in \{a, b\}^* \mid |w|_a = 2k + 1, k \geq 0\}$$

$$D_b = \{w \in \{a, b\}^* \mid |w|_b = 2n + 1, n \geq 0\}$$

Quindi

$$D = D_a \cap D_b$$

E quindi sappiamo anche

$$D_{\text{even}} = \{w \in \{a,b\}^* \mid |w| = 2h, h \geq 0\}$$

$$D_a = \{w \in \{a,b\}^* \mid |w|_a = 2k+1, k \geq 0\}$$

$$D = D_{\text{even}} \cap D_a$$

2.9 Chiusura Linguaggi Regolari e Dimostrazione Correttezza DFA

Dati due linguaggi L_1 e L_2 regolari

- $L_1 \cup L_2$
- $L_1 \cdot L_2$
- L_1^*
- L_1^R (Reversal)
- $\overline{L_1}$ (Complemento)
- $L_1 \cap L_2$

Sono tutti regolari

2.9.1 Operazioni Regolari

Siano A e B linguaggi, definiamo le operazioni regolari unione, concatenazione e star

- **Unione:** $A \cup B = \{x \mid x \in A \text{ oppure } x \in B\}$
- **Concatenazione:** $A \cdot B = \{xy \mid x \in A, y \in B\}$
- **Star:** $A^* = \{w_1 w_2 ... w_k \mid k \geq 0, w_i \in A, 1 \leq i \leq k\} = \bigcup_{n \geq 0} (A^k)$

2.9.2 Chiusura

Una classe di oggetti è chiusa rispetto ad un'operazione se l'applicazione di questa questa operazione a elementi della classe restituisce un oggetto ancora della classe

- **Chiusura Classe REG**: La classe REG è chiuso rispetto alle operazioni regolari, quindi comunque prendo 2 linguaggi regolari e faccio $\cup \cdot *$ ottengo comunque un linguaggio regolare
- **Esempio**: $\mathbb{N}$ è chiuso rispetto a $+$ e $\times$, ma non è chiuso rispetto a :

2.10 Teorema 1.25 Chiusura Classe REG Unione

La classe REG è chiuso rispetto alle operazioni regolari

2.10.1 Idea della Dimostrazione

$$L_1 \text{ e } L_2 \Rightarrow L_1 \cup L_2 \text{ regolare}$$

$$\exists M_1 \text{ t.c } L_1 = L(M_!) \text{ e } \exists M_2 \text{ t.c } L_2 = L(M_2)$$

$$M \text{ t.c } L(M) = L_1 \cup L_2$$

Va dimostrato per ogni linguaggio $\forall L_1, L_2$ regolari, $L_1 \cup L_2$ è regolare
Automa non copia, una volta processata una stringa e non la più usare, per farlo bisogna fare in parallelo, utilizzando una coppia di stati
Dobbiamo costruire un automa M che accetta se e solo se una delle due accetta (Unione di Linguaggi)

2.10.2 Dimostrazione

Sia $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ un DFA t.c. $L(M_1) = L_1$
Sia $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ un DFA t.c. $L(M_2) = L_2$

Definiamo $M = (Q, \Sigma, \delta, q_0, F)$ un DFA t.c. $L(M) = L_1 \cup L_2$
Dobbiamo dimostrare il tale che della definizione

$$Q = \{(r_1, r_2) |\ r_1 \in Q_1, r_2 \in Q_2\} = Q_1 \times Q_2$$

Con $q_0 = (q_1, q_2)$

$$F = \{(r_1, r_2) |\ r_1 \in F_1 \text{ oppure } r_2 \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$$

Di conseguenza

$$\forall a \in \Sigma \text{ e } \forall (r_1, r_2) \in Q,\ \delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

Quindi

$$w \in L(M) \Leftrightarrow w \in L_1 \cup L_2$$

$$\hat{\delta}(q_0, w) \in F \Leftrightarrow w \in L_1 \cup L_2$$

$$\Leftrightarrow \hat{\delta}_1(q_1, w) \in F_1 \text{ oppure } \hat{\delta}_2(q_2, w) \in F_2$$

Per una prova formale bisognerebbe

$$\Rightarrow \text{Vera per costruzione}$$

$$\Leftarrow \text{Si dimostra per induzione}$$

Ne consegue che $|Q| = |Q_1|Q_2|$, rispetto ad un automa creato col processo creativo, con questo metodo potrebbe avere un numero di stati maggiore ma funziona sempre

- Se i due linguaggi non sono sullo stesso alfabeto, prima di fare la costruzione dovrei estendere i due linguaggi e i due automiù sull'alfabeto unione, in modo da poter garantire che l'automa prodotto sia ben definito per ogni stato (coppia di stati) su ogni simbolo

2.11 Teorema 1.25 Bis Chiusura Classe REG Intersezione

Sebbene l'intersezione non sia una delle tre operazioni regolari base (unione, concatenazione, stella), la classe REG è chiusa anche rispetto all'intersezione
La costruzione è identica a quella dell'unione, ma cambiano gli stati finali: l'automa deve accettare se e solo se **entrambi** gli automi originali accettano
Sia $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ un DFA t.c. $L(M_1) = L_1$
Sia $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ un DFA t.c. $L(M_2) = L_2$

Definiamo $M = (Q, \Sigma, \delta, q_0, F)$ un DFA t.c. $L(M) = L_1 \cap L_2$
Dobbiamo dimostrare il tale che della definizione

$$Q = \{(r_1, r_2)|\ r_1 \in Q_1, r_2 \in Q_2\} = Q_1 \times Q_2$$

Con $q_0 = (q_1, q_2)$

$$F = \{(r_1, r_2)|\ r_1 \in F_1 \text{ e } r_2 \in F_2\} = F_1 \times F_2$$

Di conseguenza

$$\forall a \in \Sigma \text{ e } \forall (r_1, r_2) \in Q,\ \delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

$$\hat{\delta}(q_0, w) \in F \Leftrightarrow w \in L_1 \cap L_2$$

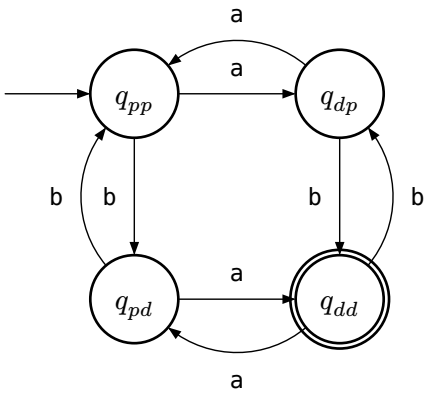
$$\Leftrightarrow \hat{\delta}_1(q_1, w) \in F_1 \text{ e } \hat{\delta}_2(q_2, w) \in F_2$$

2.11.1 Esempio 1

$$D = \{w \in \{a,b\}^* \mid |w| = 2h, h \geq 0 \text{ e } |w|_a = 2k+1, k \geq 0\}$$

Quindi devono avere anche un numero dispari di b essendo $\Sigma = \{a,b\}$

$$D = \{w \in \{a,b\}^* \mid |w|_a = 2n+1, n \geq 0 \text{ e } |w|_b = 2k+1, k \geq 0\}$$



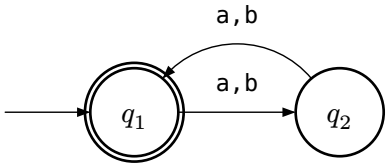
Ma sappiamo che

$$D_{\text{even}} = \{w \in \{a,b\}^* \mid |w| = 2h, h \geq 0\}$$

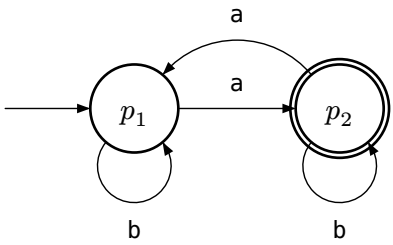
$$D_a = \{w \in \{a,b\}^* \mid |w|_a = 2k+1, k \geq 0\}$$

$$D = D_{\text{even}} \cap D_a$$

M_{even}

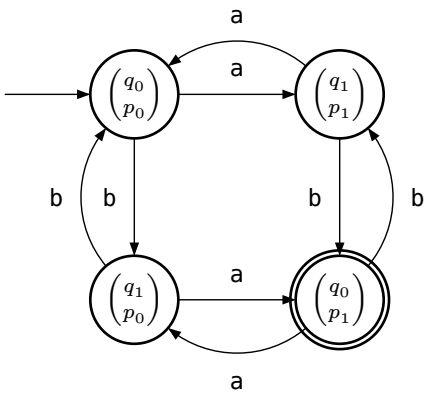


M_a



Usiamo **Lazy Construction**: Partiamo dallo stato iniziale dell'automa prodotto e via via lo completiamo considerando la transizione partendo da quello stato su tutti i simboli dell'alfabeto.

- Quando non ho più transizione uscenti finisco la costruzione.
- L'automa ha $|Q_1||Q_2|$ stati, ma può capitare che non ci siano transazioni che li collegano a partire dallo stato iniziale e quindi non li disegno, ma ci sono



Se avessimo considerato

$$D = \{w \in \{a,b\}^* \mid |w|_a = 2n+1, n \geq 0 \text{ oppure } |w|_b = 2k+1, k \geq 0\}$$

Sarebbero cambiati solo gli stati finali, e l'unico non finale sarebbe stato $\binom{q_1}{p_0}$

2.11.2 Esempio 2

- Costruire un DFA che riconosce le parole che hanno un numero dispari di b oppure un numero pari di a

$$L = \{w \in \{a,b\}^* \mid |w|_b = 2k+1, k \geq 0 \text{ oppure } |w|_a = 2h, h \geq 0\}$$

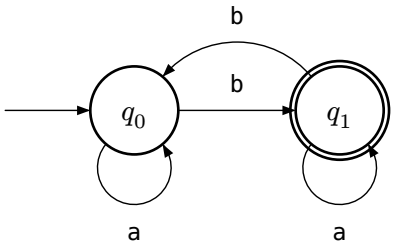
Lo posso riscrivere come

$$L_1 = \{w \in \{a,b\}^* \mid |w|_b = 2k+1, k \geq 0\}$$

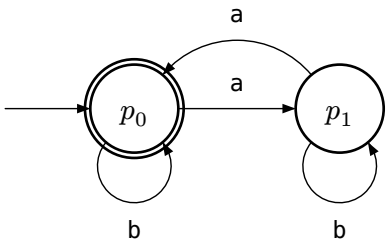
$$L_2 = \{w \in \{a,b\}^* \mid |w|_a = 2h, h \geq 0\}$$

$$L = L_1 \cup L_2$$

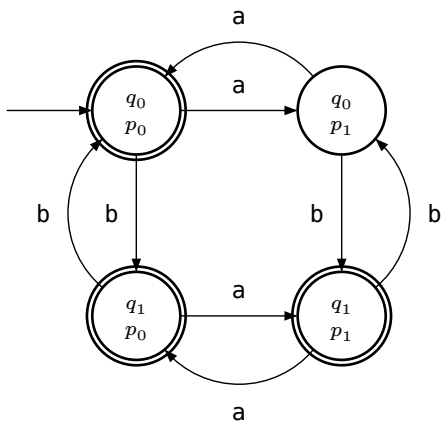
M_1



M_2



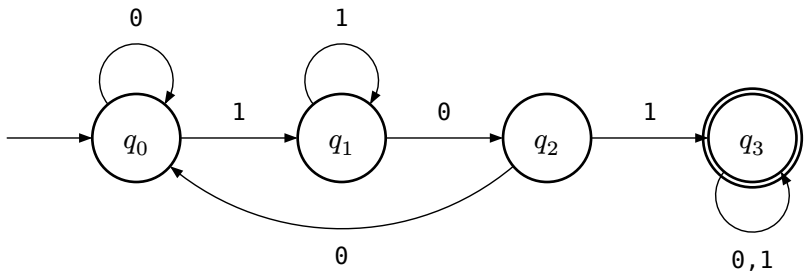
Uso Lazy Construction M



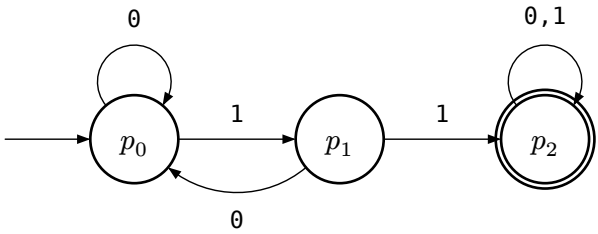
2.11.3 Esempio 3

- Progettare un DFA che riconosce tutte le stringhe su {0,1} che contengono 101 oppure 11 come sottostringa

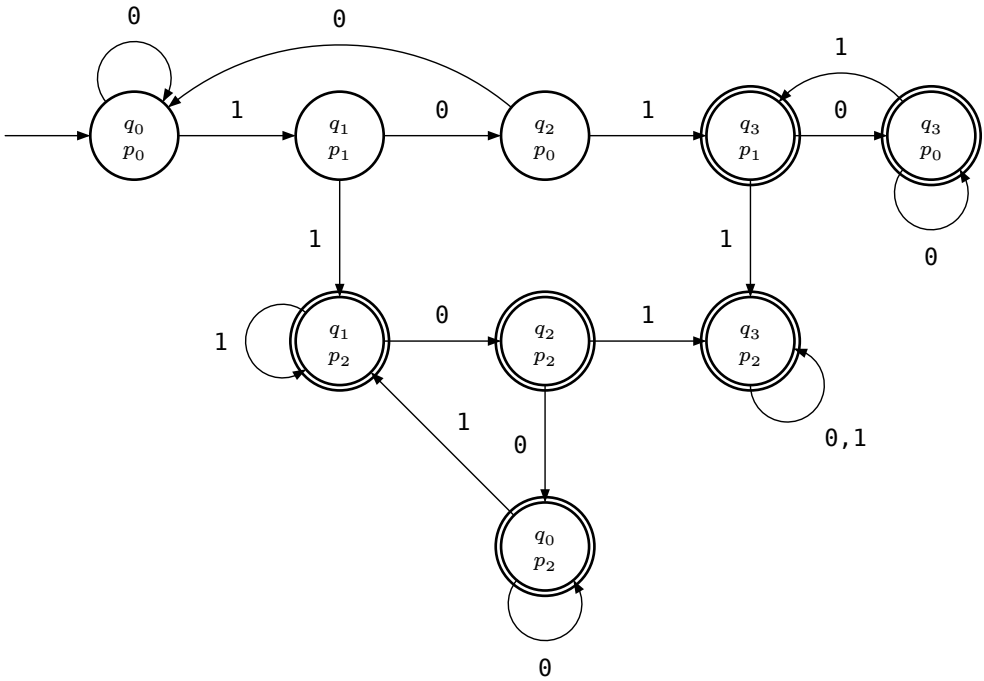
M_1: 101 come fattore



M_2: 11 come fattore



Applichiamo Lazy Construction

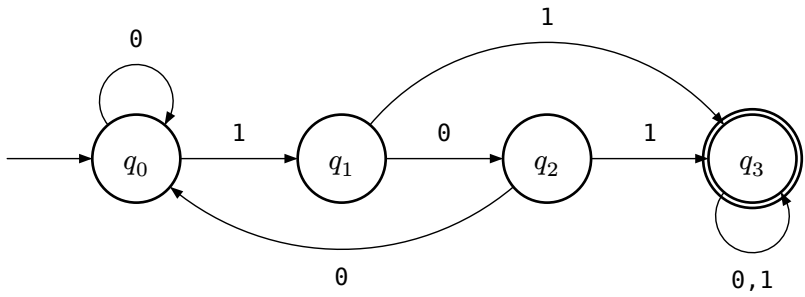


$$|Q| = |Q_1||Q_2|$$

Sono disegnati solo gli stati raggiugiili dallo stato iniziale, ci sono stati non raggiungibili come $\binom{q_0}{p_1}$, ma sono comunque presenti e definiti nella funzione di transizione

$$L(M) = L_1 \cup L_2$$

Uso metodo creativo



Abbiamo più stati rispetto al metodo creativo, ma l'automa è corretto grazie al Teorema 1.25

2.12 Teorema (4.5 HUM) Chiusura Classe REG Complemento

La classe REG è chiusa rispetto all'operazione di complemento

$$\overline{\mathrm{L}}_1 = \Sigma^* - L_1$$

Creo una copia di M_1 che risolve una condizione e poi complemento gli stati per ottenere quindi il complemento di L_1.
Si prova attraverso la dimostrazione costruttiva sull'unione.
Un'altra prova è rispetto la chiusura all'intersezione, non costruttiva.
Si possono applicare le leggi di DeMorgan

$$L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$$

- L_1, L_2 regolari
- $\overline{L_1}, \overline{L_2}$ regolari
- $\overline{L_1} \cup \overline{L_2}$ regolari
- $\overline{\overline{L_1} \cup \overline{L_2}}$ regolari
- $L_1 \cap L_2$ regolari

2.13 Teorema 1.26 Chiusura Classe REG Concatenazione

- **Idea**: L'automa accetta solo se l'input può essere diviso in due parti, la prima accettata dal primo automa e la seconda parte dal secondo automa, quindi una computazione in parallelo

2.14 Organizzazione Riposta DFA con Teorema Chiusura

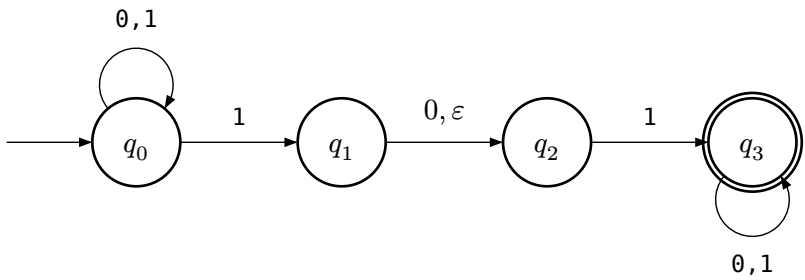
1) Scrivete esplicitamente che fornite un DFA che riconosce il linguaggio sfruttando le proprietà di chiusura (e dite quale). Disegnate i 2 automi da cui partite e riportate le quintuple, esplicitando anche le tabelle di transizione

2) Riportate la definizione dell'automa prodotto, cioè la quintipla così come data nella prova del Teorema 1.25, specificando chi è lo stato iniziale e quali sono gli stati finali di Q1xQ2

3) Per la funzione di transizione, dopo aver riportato la definizione nel 2) dovete esplicitarla: o scrivete la tabella di transizione o il diagramma di stato. Questo servirà quando cercheremo di fornire (nelle prossime lezioni) una forma al linguaggio accettato, che diventa complicato capire leggendo la tabella di transizione. Nel fare il diagramma di stato, dichiarate che usate la lazy construction, quindi che rappresentate solo gli stati raggiungibili dallo stato iniziale. Eventuali stati di Q1xQ2 non disegnati, sono stati su cui la funzione di transizione è definita ma non viene esplicitata perchè non contribuiscono alla determinazione del linguaggio accettato dall'automa.

3 Automi Finiti Non Deterministici

- Consentiamo ε-transizioni, aggiungendo la stringa vuota ε
  - La computazione si dive in più copie parallele per ogni scelta
  - Con ε, quindi, una computazione resta nello stato ed un'altra segue la transizione, ma la stringa rimane sempre unica
- Eliminiamo l'obbligo che per ogni stato esista esattamente una transizione per ogni simbolo dell'alfabeto



Quindi una computazione resta in q_1 dove non leggo 1, mentre in parallelo un'altra va in q_3 dove posso leggere 1, quindi esplorazione di tutti i possibili cammini.

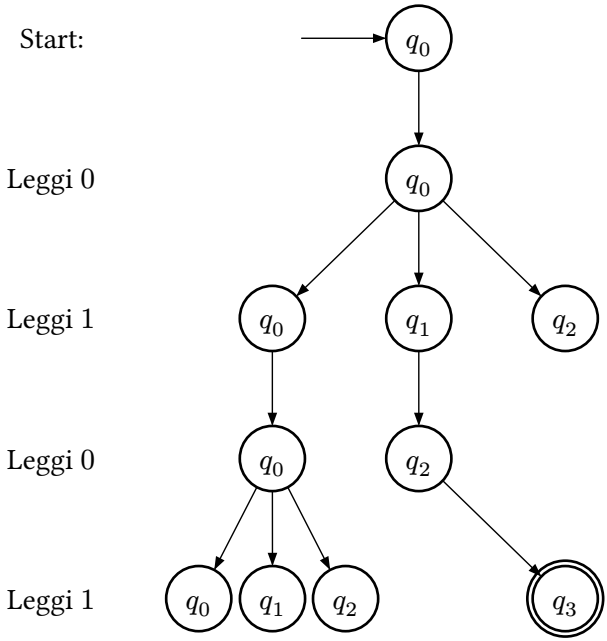
Con ε non consumo l'input.

Dove non ci sono transizione per alcuni simboli dell'alfabeto abbiamo $\emptyset$.

Una computazione è accetta se almeno un cammino per quella stringa è accettato.

Si può sempre esprimere un NFA come un DFA0

- Albero della computazione rispetto l'automa precedente



3.1 Linguaggio Accettato o Riconosciuto NFA

Un NFA accetta tutte e sole le stringhe di Σ^* per le quali esiste un cammino di accettazione, ovviamente non è detto che esista sempre un cammino e che sia unico

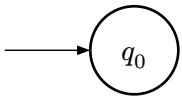
- Accetto la stringa vuota se lo stato iniziale è finale, oppure se esiste un path dallo stato iniziale ad uno stato finale ottenuto seguendo esclusivamente transizioni etichettate con la stringa vuota

Sia $A = \{Q, \Sigma, \delta, q_0, F\}$ un automa non deterministico, il linguaggio accettato o riconosciuto da A è

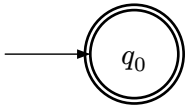
$$L(A) = \left\{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset \right\}$$

3.2 Semplici NFA

- $L(M) = \emptyset$

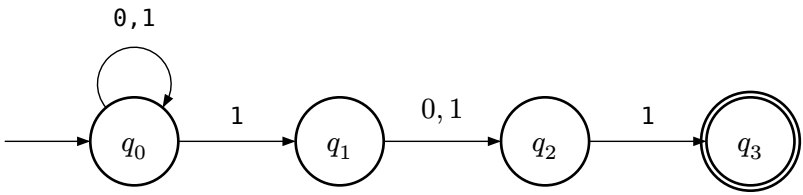


- $L(M) = \{\varepsilon\}$



3.3 Esempio NFA

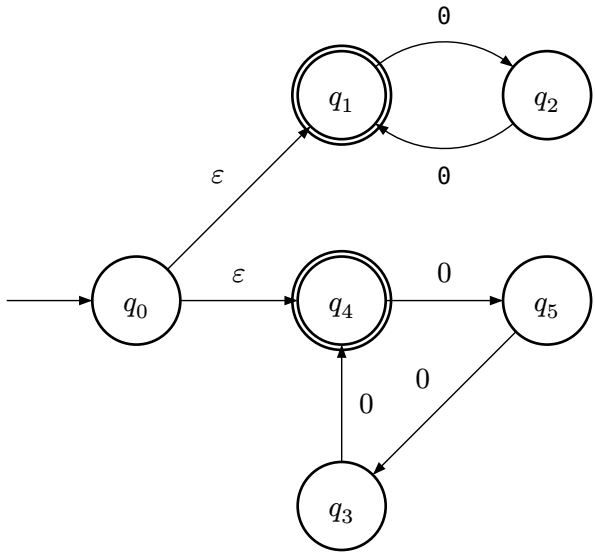
3.3.1 Esempio 1



$$L(M) = \{w \in \{0,1\}^+ \mid w \text{ ha 1 nella terza posizione dalla fine}\}$$

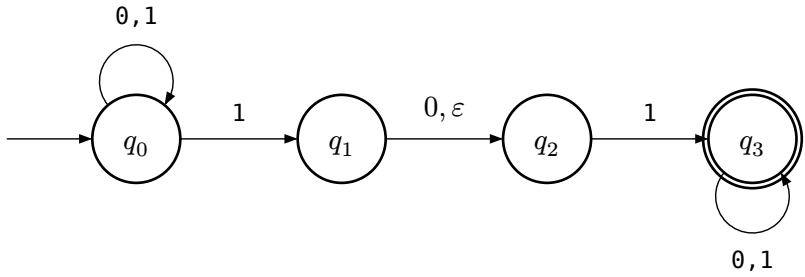
$$L(M) = \{w \in \{0,1\}^+ \mid w = xly, x \in \{0,1\}^*, y \in \{0,1\}^2\}$$

3.3.2 Esempio 2



$L(M) =$ accetta tutte le stringhe della forma 0^k con k multiplo di 2 o 3

3.3.3 Esempio 3



$$N = (Q, \Sigma, \delta, q_1, F)$$

$$Q = \{q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$\left(\begin{array}{c|ccc} & 0 & 1 & \varepsilon \\ \hline q_1 & \{q_1\} & \{q_1, q_2\} & \emptyset \\ q_2 & \{q_3\} & \emptyset & \{q_3\} \\ q_3 & \emptyset & \{q_4\} & \emptyset \\ q_4 & \{q_4\} & \{q_4\} & \emptyset \\ & & & \end{array}\right)$$

q_1 è lo stato iniziale

$$F = \{q_4\}$$

3.4 Definizione Formale NFA

Un automa a stati finiti non deterministico è una quintupla
$A = (Q, \Sigma, \delta, q_0, F)$

- Q: Insieme finito degli stati
- Σ: Alfabeto finito
- $q_0 \in Q$: Stato iniziale
- $\delta : (Q \times \Sigma_\varepsilon) \to P(Q)$: Funzione di transizione con $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ che traduce il comportamento non deterministico dell'automa, $P(Q)$ è l'insieme potenza, ovvero la collezione di tutti i possibili insiemi
- $F \subseteq Q$: Insieme degli stati finali

3.5 Epsilon-Chiusura (HUM 2.5.3-2.5.4)

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito non deterministico, sia $q \in Q$.
La ε-chiusura $E(q)$ di q è un sottoinsieme di Q definito ricorsivamente come segue:

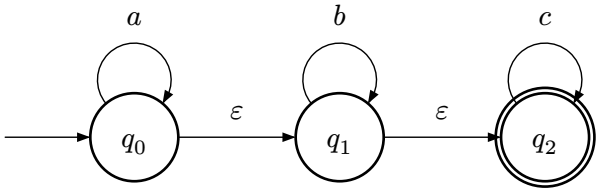
- Passo base: $q \in E(q)$
- Passo ricorsivo: $\forall p \in E(q), \delta(p, \varepsilon) \subseteq E(q)$

Quindi sia $R \subseteq Q$, la ε-chiusura $E(R)$ di R è

$$E(R) = \cup_{q \in R} E(q)$$

- ε-chiusura non è mai $\emptyset$, essendo l'insieme con almeno il primo stato.
- Rappresenta tutti gli stati raggiungibili con le ε-transizioni
- Per ogni stato in R, considero le ε-chiusure di ogni stato

3.5.1 Esempio ε-chiusura



Calcolo $E(q) =?$

- $q_0 \in E(q_0)$, quindi $E(q_0) = \{q_0,$
- $\delta(q_0, \varepsilon) = \{q_1\} \to q_1 \in E(q_0)$, quindi $E(q_0) = \{q_0, q_1$
- $\delta(q_1, \varepsilon) = \{q_2\} \to q_2 \in E(q_0)$, quindi $E(q_0) = \{q_0, q_1, q_2$
- $\delta(q_2, \varepsilon) = \emptyset$, quindi $E(q_0) = \{q_0, q_1, q_2\}$

Quindi considero la δ e ε-chiusura

- Altro metodo
- $E(q_0) = \{q_0\} \cup E(q_1)$
- $E(q_1) = \{q_1\} \cup E(q_2)$
- $E(q_2) = \{q_2\}$

Quindi è equivalente

- $E(q_0) = \{q_0, q_1, q_2\}$
- $E(q_1) = \{q_1, q_2\}$
- $E(q_2) = \{q_2\}$

3.6 Funzione di Transizione Estesa NFA

Sia $A = (Q, \Sigma, \delta, q_0, F)$ un automa finito non deterministico, la funzione di transizione estesa $\hat{\delta} : Q \times \Sigma^* \to P(Q)$ è definita ricorsivamente come segue

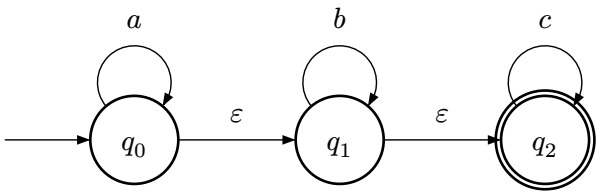
- Passo base: $\forall q \in Q,\ \ \hat{\delta}(q, \varepsilon) = E(q)$
- Passo ricorsivo: $\forall q \in Q, w \in \Sigma^*, a \in \Sigma$,

$$\hat{\delta}(q, wa) = E\Big(\cup_{p \in \hat{\delta}(q,w)} \delta(p, a)\Big) =$$

$$\cup_{p \in \hat{\delta}(q,w)} E(\delta(p, a))$$

Quindi per ogni possibili cammino in cui lo stato segue una transizione in a

3.6.1 Esempio Funzione di Transizione Estesa



- $E(q_0) = \{q_0, q_1, q_2\}$
- $E(q_1) = \{q_1, q_2\}$
- $E(q_2) = \{q_2\}$

Calcolo $\hat{\delta}(q_0, ca)$
- $\hat{\delta}(q_0, ca) = E\Big(\cup_{p \in \hat{\delta}(q_0,c)} \delta(p, a)\Big) =$
- $\hat{\delta}(q_0, c) = E\Big(\cup_{p \in \hat{\delta}(q_0,\varepsilon)} \delta(p, c)\Big) =$
- $\hat{\delta}(q_0, \varepsilon) = E(q_0) = \{q_0, q_1, q_2\}$

Quindi risalendo con la ricorsione
- $\hat{\delta}(q_0, c) = E\Big(\cup_{p \in \hat{\delta}(q_0,\varepsilon)} \delta(p, c)\Big) =$
- $= E(\delta(q_0, c) \cup \delta(q_1, c) \cup \delta(q_2, c)) =$
- $E(\emptyset \cup \emptyset \cup \{q_2\}) = E(q_2) = \{q_2\}$

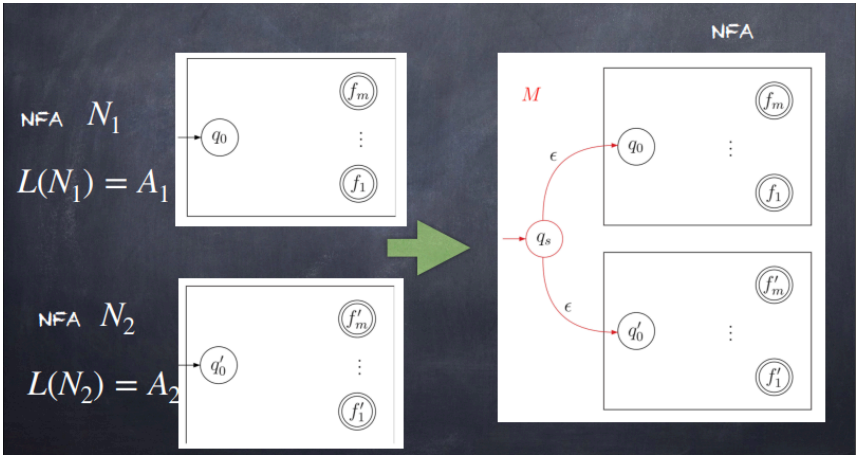
Quindi risalendo nella ricorsione

- $\hat{\delta}(q_0, ca) = E\Big(\cup_{p \in \hat{\delta}(q_0, c)} \delta(p, a)\Big) =$
- $E(\delta(q_2, a)) = E(\emptyset) = \emptyset$

Questo automa accetta $L = \{a^i b^j c^k \mid i, j, k \geq 0\}$

3.7 Teorema 1.45 Chiusura Classe REG Unione NFA

Dato due qualsiasi linguaggi regolari A_1 e $A_2 \Rightarrow A_1 \cup A_2$ è



Dimostrazione:
Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed
$N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2
Costruiamo $N = (Q, \Sigma, \delta, q, F)$ per riconoscere $A_1 \cup A_2$

- $Q = \{q_0\} \cup Q_1 \cup Q_2$

Gli stati di N sono tutti gli stati di N_1 e N_2 con l'aggiunta di un nuovo stato iniziale q_0

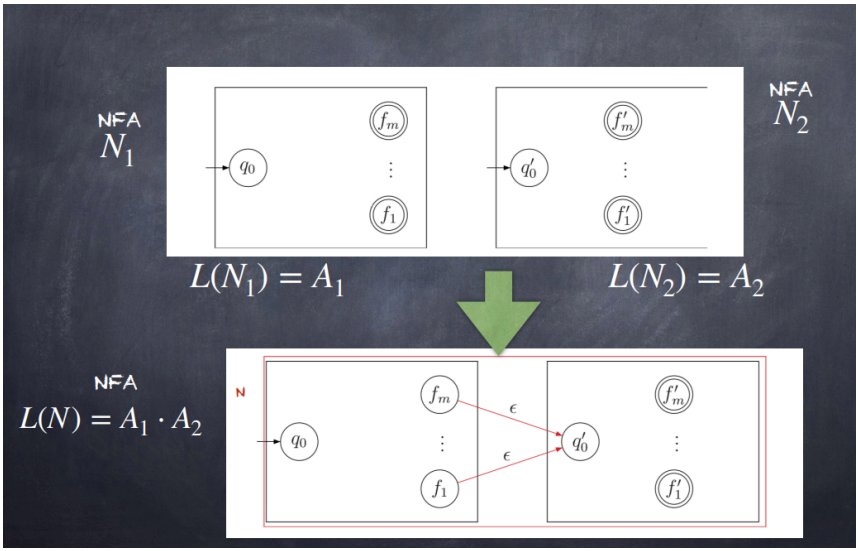
- Stato q_0 è lo stato iniziale di N
- Insieme degli stati accettati $F = F_1 \cup F_2$

Gli stati accettati di N sono tutti gli stati accettati di N_1 e N_2, in questo modo N accetta se N_1 accetta o N_2 accetta

- Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\varepsilon$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \in Q_1 \\ \delta_2(q, a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ e } a = \varepsilon \\ \emptyset & q = q_0 \text{ e } a \neq \varepsilon \end{cases}$$

3.8 Teorema 1.47 Chiusura Classe REG Concatenazione NFA



Dimostrazione:
Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1 ed
$N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ che riconosce A_2
Costruiamo $N = (Q, \Sigma, \delta, q_1, F_2)$ per riconoscere $A_1 \cdot A_2$

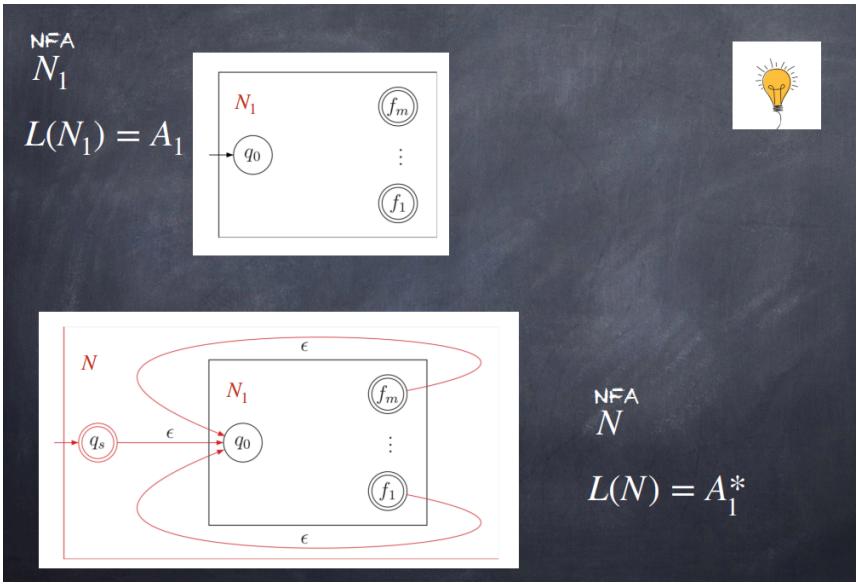
- $Q = Q_1 \cup Q_2$

Gli stati di N sono tutti gli stati di N_1 e N_2

- Stato q_1 è uguale allo stato iniziale di N_1
- Gli stati accettati F_2 sono uguali agli stati accettati di N_2
- Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\varepsilon$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \text{ e } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1 \text{ e } a = \varepsilon \\ \delta_2(q,a) & q \in Q_2 \end{cases}$$

3.9 Teorema 1.49 Chiusura Classe REG Chiusura di Kleene



Dimostrazione:
Sia $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ che riconosce A_1
Costruiamo $N = (Q, \Sigma, \delta, q_0, F)$ per riconoscere A_1^*

- $Q = \{q_0\} \cup Q_1$

Gli stati di N sono gli stati di N_1 più un nuovo stato iniziale

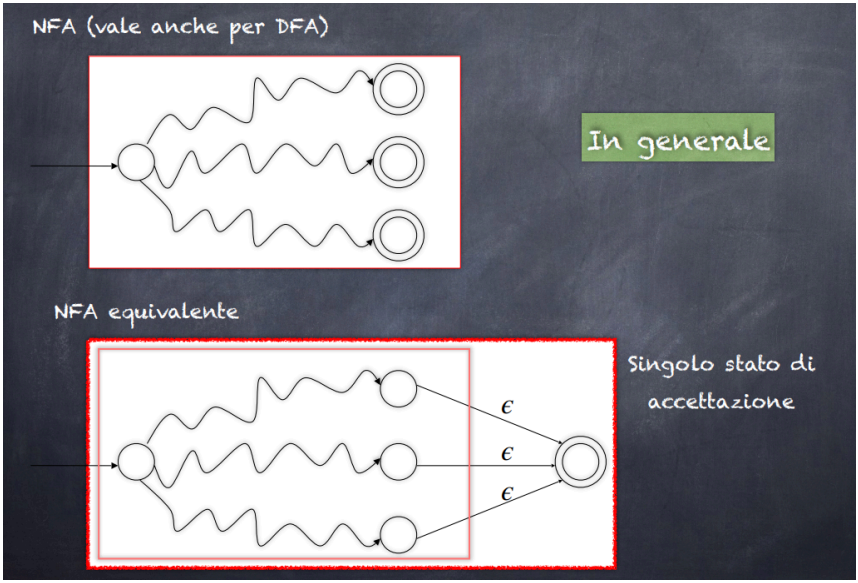
- Stato q_0 nuovo stato iniziale
- $F = \{q_0\} \cup F_1$

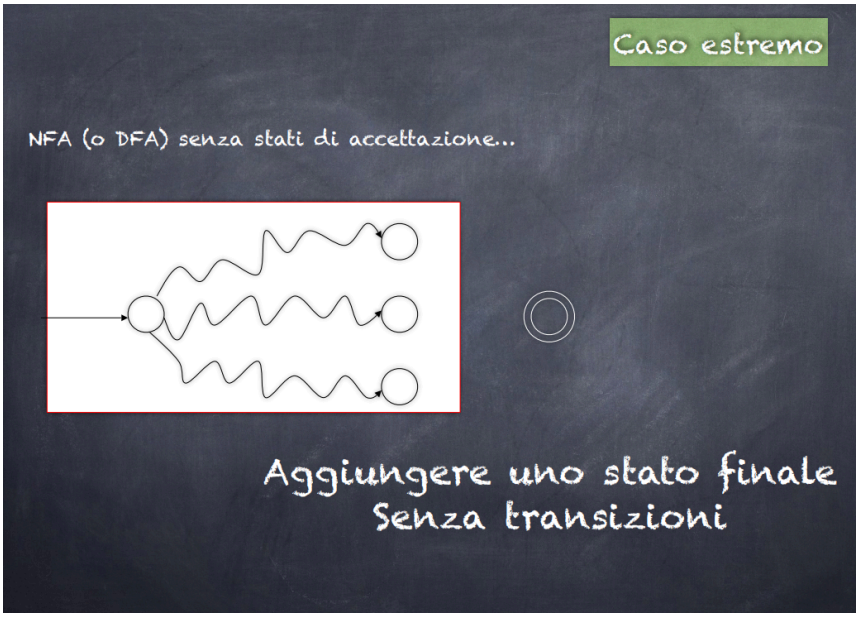
Gli stati accettati sono i vecchi stati accettati più il nuovo stato iniziale

- Definiamo δ in modo che per ogni $q \in Q$ e ogni $a \in \Sigma_\varepsilon$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \text{ e } q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \text{ e } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & q \in F_1 \text{ e } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ e } a = \varepsilon \\ \emptyset & q = q_0 \text{ e } a \neq \varepsilon \end{cases}$$

3.10 Chiusura Classe REG Reverse





Sia $M = (Q, \Sigma, \delta, q_0, F)$ che riconosce L.
Supponiamo di voler progettare un automa finito $R = (Q', \Sigma, \delta', q'_0, F')$ che riconosce il reverse di L (per non perdere di generalità, sia R un NFA). Poiché $\mathrm{Rev}(L)$ deve riconosce le stesse stringhe di L ma lette al contrario, l'idea è quella di far partire la computazione dalla fine (stati finali) e di andare a ritroso verso l'inizio (stato iniziale).
Formalmente, per ogni $p \in \delta(q_a)$, per ogni $q \in Q, a \in \Sigma$ si ha che $q \in \delta'(p, a)$.
In questo modo stiamo "invertendo le frecce" di M.

Un problema può essere il fatto che la computazione per partire dalla fine, dovrebbe partire dagli stati finali, ma lo stato iniziale degli automi è unico. Quindi l'automa R avrà gli stati di M, con l'aggiunta di un nuovo stato che sarà iniziale e dal quale, con epsilon transizioni, andrò negli stati finali. Lo stato iniziale di M sarà lo stato finale di R.
Formalmente, $F' = \{q_0\}$ mentre q'_0 sarà un nuovo stato, cioè $Q' = Q \cup \{q'_0\}$.
Inoltre $\delta'(q'_0, \varepsilon) = F$, mentre $\delta'(q'_0, a) = \emptyset$, per a diverso dalla stringa vuota.
Non è necessario provare rigorosamente che $\delta'(q'_0, \varepsilon) \in F$ se e solo se $\delta'(q'_0, w^R) \in F$, perché segue dalla costruzione. Nelle slides della lezione c'è un esempio di costruzione.

3.11 Chiusura Classe REG Prefissi e Suffissi

- Prefissi di L: $\{x \in \Sigma^* \mid \exists y \in \Sigma^* : xy \in L\}$
- Suffissi di L: $\{y \in \Sigma^* \mid \exists x \in \Sigma^* : xy \in L\}$

Si può dimostrare che se L è regolare allora:
- Pref(L) è regolare
- Suff(L) è regolare

3.11.1 Chiusura Classe REG Prefissi

Innanzitutto, ricordiamo che dato un linguaggio L, indichiamo con $\text{Pref}(L) = \{x \in \Sigma^* \mid w = xy \in L, x, y \in \Sigma^*\}$.

Seguiamo il ragionamento fatto durante la lezione e supponiamo che l'automa $M = (Q, \Sigma, \delta, q_0, F)$ che riconosce L sia un NFA in cui non ci sono archi uscenti dagli stati finali. Questa ipotesi è facile da soddisfare aggiungendo all'automa un nuovo stato finale in cui entro leggendo la stringa vuota a partire da ogni stato finale di M.
Formalmente, consideriamo un automa $M' = (Q', \Sigma, \delta', q_0, \{q_F\})$ che riconosce L, con $Q' = Q \cup \{q_F\}$ dove $\delta'(q, a) = \delta(q, a)$, per ogni q in Q, a in Σ, e in più, per ogni q in F si ha che $q_F \in \delta'(q, \varepsilon)$.
Senza perdere di generalità, essendo un NFA, possiamo eliminare tutti gli stati pozzo, cioè quelli che non consentiranno di raggiungere gli stati finali.

Costruiamo ora $P = (Q', \Sigma, \delta'', q_0, \{q_F\})$ dove $\delta''(q, a) = \delta'(q, a)$ per ogni $q \in Q', a \in \Sigma$ e inoltre per ogni $q \in Q', q_F \in \delta''(q, \varepsilon)$, se q raggiunge uno stato finale. Non è necessario provare rigorosamente che L(P)=Pref(L), cioè l'insieme dei prefissi di L, perché se w=xy, e w è accettata, allora $\hat{\delta}(q_0, w) = \hat{\delta}(q_0, xy) = \hat{\delta}\left(\hat{\delta}(q_0, x), y\right)$.
Sia $t \in \hat{\delta}(q_0, x)$. Allora $\hat{\delta}(t, y) = \{q_F\}$ se e solo se $\{q_F\} \in \hat{\delta}'(q_0, x)$, perché c'è la epsilon transizione da ogni stato come t che viene raggiunto da $\hat{\delta}'(q_0, x)$ verso q_F.
Inoltre, l'ipotesi garantisce che se raggiungo q_F non esco verso nessun altro stato. Se così non fosse, non garantirei di riconosce solo i prefissi di L, ma potrei continuare e leggere altre stringhe.

3.11.2 Chiusura Classe REG Suffissi

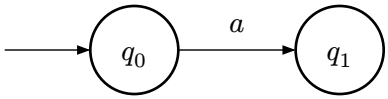
Innanzitutto, ricordiamo che dato un linguaggio L, indichiamo con Suff(L)= $\{y \in \Sigma^* \mid w = xy \in L, x, y \in \Sigma^*\}$.

Anche per questo esercizio, seguiamo il ragionamento fatto durante la lezione e supponiamo che l'automa $M = (Q, \Sigma, \delta, q_0, F)$ che riconosce L sia un NFA in cui non ci sono archi entranti nello stato iniziale. Se infatti questo non accadesse, potremmo trovarci in una situazione simmetrica a quella dei prefissi, riconoscendo stringhe non suffissi di L.
Questa ipotesi è facile da soddisfare aggiungendo all'automa un nuovo stato iniziale da cui usciamo leggendo la stringa vuota verso lo stato iniziale di M.
Formalmente, consideriamo un automa $M' = (Q', \Sigma, \delta', q'_0, F\}$ che riconosce L, con $Q' = Q \cup \{q'_0\}$ dove $\delta'(q, a) = \delta(q, a)$, per ogni q in Q, a in Σ, mentre $\{q_0\} \in \delta'(q'_0, \varepsilon)$ e $\delta'(q'_0, a) = \emptyset$.

Costruiamo ora $S = (Q', \Sigma, \delta'', q'_0, F)$ dove $\delta''(q, a) = \delta'(q, a)$ per ogni $q \in Q'$, $a \in \Sigma$ e inoltre per ogni $q \in Q', q \in \delta''(q'_0, \varepsilon)$, se q è raggiungibile dallo stato iniziale.
Non è necessario provare rigorosamente che L(S)=Suff(L), cioè l'insieme dei suffissi di L, perché se w=xy, e w è accettata, allora $\hat{\delta}'(q_0, \varepsilon) \subseteq F$, perché c'è la epsilon transizione da ogni stato raggiunto da $\hat{\delta}'(q_0, \varepsilon)$ ad uno stato finale.
Quindi con la epsilon transizione, da q_0 vado in $\hat{\delta}'(q_0, \varepsilon)$ a partire dal quale so che posso leggere anche y e arrivare in uno stato finale, quindi accettando y. L'ipotesi che sullo stato iniziale non ci siamo archi entranti, garantisce che posso solo "saltare" la x nella lettura

4 Ogni NFA è un DFA e Viceversa

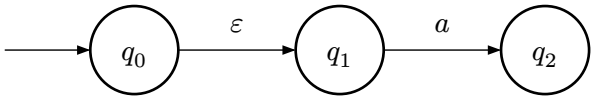
Per i DFA $\hat{\delta}(q,a) = \delta(q,a),\ \ a \in \Sigma$ è sempre vero



$$\delta(q_0,a) = q_1$$

$$\hat{\delta}(q_0,a) = \delta\Big(\hat{\delta}(q_0,\varepsilon),a\Big) = \delta(q_0,a)$$

Per gli NFA, invece, $\hat{\delta}(q,a) = \delta(q,a),\ \ a \in \Sigma$ non è vero in generale



$$\delta(q_0,a) = \emptyset$$

$$\hat{\delta}(q_0,a) = \hat{\delta}(q_0,\varepsilon a) =$$

$$= E\Big(\cup_{p\in\hat{\delta}(q_0,\varepsilon)}\ \delta(q_0,a)\Big) =$$

$$= E(\delta(q_0,a) \cup \delta(q_1,a)) =$$

$$= E(\emptyset \cup q_2) = \emptyset \cup \{q_2\} = \{q_2\}$$

4.1 Ogni DFA è un NFA

$$\text{DFA } M = \{Q, \Sigma, \delta, q_0, F\}, \quad \delta : Q \times \Sigma \to Q$$

$$\text{NFA } N = \{Q, \Sigma_\varepsilon, \delta_N, q_0, F\}, \quad \delta_N : Q \times \Sigma_\varepsilon \to P(Q)$$

$$L(N) = L(M)$$

$$\delta(q, a) = p \Rightarrow \delta_{N(q,a)} = \{p\}$$

Quindi considero un singolo stato di un DFA come il singleton di quello stato nel NFA, a parità di linguaggio

$$L(M) = \left\{ w \in \Sigma^* \mid \hat{\delta}_{M(q_0,w)} \in F \right\} =$$

$$\left\{ w \in \Sigma^* \mid \hat{\delta}_{N(q_0,w)} \cap F \neq \emptyset \right\} = L(N)$$

4.2 Teorema 1.39 NFA e DFA equivalenti

Per ogni automa finito non deterministico A esiste un automa finito deterministico B tale che $L(A) = L(B)$

- Se nel NFA parto da q_0 ma ci sono anche ε-transizioni, allora anche quegli stati sono iniziali
- Se nel NFA termino in uno stato finale allora nel DFA ogni stato che contiene uno stato finale del NFA rappresenta un cammino di accettazione

Dimostrazione:

$$\forall \text{ NFA } N = \{Q_N, \Sigma, \delta_N, q_N, F_N\}$$

$$\exists \text{ DFA } M = \{Q_M, \Sigma, \delta_M, q_M, F_M\}$$

$$\text{t.c. } L(N) = L(M)$$

Dim:

$$Q_M = P(Q_N), q_M = E(q_n)$$

$$F_M = \{R \in Q_M \mid R \cap F_N \neq \emptyset\}$$

$$\delta_M(R, a) = E(\cup_{r \in R} \delta_N(r, a))$$

Ora dimostriamo che $L(N) = L(M)$

$$\forall w \in \Sigma^*, \hat{\delta}(q_M, w) = \hat{\delta}(q_N, w)$$

- Passo base:

Sia $w = \varepsilon$

$$\hat{\delta}(q_M, \varepsilon) =^{\text{def DFA}} q_M =^{\text{Costruzione}} E(q_N) =^{\text{def NFA}} \hat{\delta}(q_N, \varepsilon)$$

- Passo induttivo:

Sia $w = xa, x \in \Sigma^*, a \in \Sigma$
Supponiamo $\hat{\delta}_M(q_M, x) = \hat{\delta}_N(q_N, x)$

$$\hat{\delta}_M(q_M, xa) \overset{\text{def DFA}}{=} \delta_M\Big(\hat{\delta}_M(q_M, x), a\Big) \overset{\text{Costruzione}}{=}$$

$$E\Big(\cup_{r \in \hat{\delta}_M(q_M, x)} \delta_N(r, a)\Big)$$

Applico ipotesi induttiva

$$= E\Big(\cup_{r \in \hat{\delta}_N(q_N, x)} \delta_N(r, a)\Big) = \hat{\delta}_N(q_n, xa)$$

4.2.1 Corollario Teorema 1.39

Un linguaggio è regolare se e solo se esiste un automa finito non deterministico che lo riconosce

- Quindi se esiste un NFA, un DFA o usando le proprietà di chiusura

5 Espressioni Regolari

Teorema di Kleene: Un linguaggio è regolare se e solo se esiste un espressione regolare che lo rappresenta, quindi vale il viceversa

5.1 Definizione Formale Espressioni Regolari

- Passo Base:

Per ogni $a \in \Sigma$, a è un espressione regolare
ε è un espressione regolare
$\emptyset$ è un espressione regolare

- Passo Ricorsivo:

Se E_1, E_2 sono espressioni regolari, allora
(E_1) è un espressione regolare
$(E_1 \cup E_2)$ è un espressione regolare
$(E_1 E_2)$ è un espressione regolare
(E_1^*) è un espressione regolare

5.2 Precedenza Operatori

Operatore	Precedenza
$*$	1
.	2
$\cup$	3

5.3 Definizione Ricorsiva Linguaggi Rappresentati REG

Data un espressione regolare E, indicheremo con $L(E)$ il linguaggio che essa rappresenta, definito come segue:

- Passo base:

Per ogni $a \in \Sigma, L(A) = \{a\}$
$L(\varepsilon) = \{\varepsilon\}$
$L(\emptyset) = \emptyset$

- Passo Ricorsivo:

Se E_1, E_2 sono espressioni regolari allora
$L((E_1)) = L(E_1)$ è un espressione regolare
$L(E_1 \cup E_2) = L(E_1) \cup L(E_2)$ è un espressione regolare
$L(E_1 E_2) = L(E_1) L(E_2)$ è un espressione regolare
$L(E_1^*) = L(E_1)^*$ è un espressione regolare

5.4 Proprietà Algebriche RE

- $E_1 \cup E_2 = E_2 \cup E_1$
- $(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$
- $\emptyset \cup E_1 = E_1$
- $(E_1 E_2) E_3 = E_1 (E_2 E_3)$
- $\emptyset E_1 = E_1 \emptyset = \emptyset$
- $\varepsilon E_1 = E_1 \varepsilon = E_1$
- $E_1 (E_2 E_3) = E_1 E_2 \cup E_1 E_3$
- $(E_1 \cup E_2) \cup E_3 = E_1 E_3 \cup E_2 E_3$
- $E_1 \cup E_1 = E_1$
- $((E_1)^*)^* = E_1^*$
- $E^+ = E E^*$
- $(E_1 \cup E_2)^* = (E_1^* E_2^*)^*$

6 Pumping Lemma

Se A è un linguaggio regolare, allora $\exists p > 0$ tale che $\forall$ stringa $s \in A$ di lunghezza almeno p (cioè $|s| \geq p$), esistono x, y, z tali che $s = xyz$ e valgono le seguenti condizioni:

- $xy^i z \in A, \forall i \geq 0$
- $|y| \geq 1$
- $|xy| \leq p$

6.1 Tecnica Per Provare la Non Regolarità

- Supponiamo A regolare (quindi vale PL)
- Deve esistere $p > O$ tale che tutte le stringhe di A di lunghezza maggiore o uguale a p possono essere iterate (non sappiamo p chi sia, ma esiste)

3. Troviamo una stringa s di A che ha lunghezza maggiore o uguale a p che però non può essere iterata considerando TUTTI i possibili modi di fattorizzarla in x, y, z (con le condizioni)
4. Per ogni fattorizzazione, troviamo i tale che $xy^i z$ non appartiene a A. Assurdo!

7 Macchine di Turing

7.1 Descrizione Informale

- Utilizza un nastro semi-infinito
- Presenta una testina di lettura e scrittura che può muoversi avanti e dietro
- Ha due stati speciali q_{accept} e q_{reject} che hanno effetto immediato

7.2 Definizione Formale MdT

Un macchina di Turing deterministica è una settupla

$$\left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$$

- Q: Insieme finito degli stati
- Σ: Alfabeto dei simboli in input, con $\sqcup \notin \Sigma$
- Γ: Alfabeto finito dei simboli di nastro, con $\sqcup \in \Gamma, \Sigma \subset \Gamma,\ L, R \notin \Gamma$
- δ: Funzione di transizione

$$\left(Q \setminus \left\{q_{\text{accept}}, q_{\text{reject}}\right\}\right) \times \Gamma \to Q \times R \times \{L, R\}$$

- $q_0 \in Q$: Stato iniziale
- $q_{\text{accept}} \in Q$: Stato di accettazione
- $q_{\text{reject}} \in Q$: Stato di rifiuto, $q_{\text{accept}} \neq q_{\text{reject}}$

7.3 Funzione di Transizione MdT

Se $\delta(q, \gamma) = (q', \gamma', d)$ sappiamo che $q, q' \in Q, \gamma, \gamma' \in \Gamma, d \in \{L, R\}$

7.4 Diagramma di Stato

Grafo i cui nodi sono gli stati della macchina e le etichette sugli archi hanno la seguente forma

$$\text{simbolo} \to \text{simbolo}, d$$

Con il simbolo $\in \Gamma$ e $d \in \{L, R\}$

7.5 Computazione di una Mdt Informale

- Inizia dallo stato iniziale q_0
- Con l'input $w \in \Sigma^*$ posizionato sulla parte più a sinistra del nastro, e con la testina posizionata sulla cella più a sinistra del nastro
- Se input ε, allora il nastro contiene solo $\sqcup$
- La computazione di M procede fino a quando non viene raggiunto uno stato di accettazione o rifiuto. Se nessuno dei due stati viene raggiunto, la computazione di M continua per sempre
- Dato che Σ non contiene $\sqcup$ all'inizio della computazione il primo simbolo $\sqcup$ segna la fine della computazione
- La computazione termina se ha raggiunto q_{accept} e q_{reject}, oppure può non terminare

7.6 Configurazione di una MdT

Una configurazione C di una MdT $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ è una stringa $C = uqv \in \Gamma^* Q \Gamma^S$, con $\Gamma^S = \Gamma^*(\Gamma \setminus \{\sqcup\}) \cup \{\sqcup, \varepsilon\}$ con

- $q \in Q$: Stato corrente di M
- $uv \in \Gamma^*$ è il contenuto del nastro
  - Convenzione nel eliminare tutti i simboli $\sqcup$ che seguono ultimo carattere di v se $v \neq \varepsilon$
  - Tutti i simboli $\sqcup$ che seguono u altrimenti
- Testina posizionata sul primo simbolo di v se $v \neq \varepsilon$, su $\sqcup$ altrimenti

Una configurazione C di una MdT M si dice:

- **Iniziale**: (Con input w) se $C = q_o w, w \in \Sigma^*$
- **Di arresto**: Se $C = uqv, u, v \in \Gamma^*$ e $q \in \left\{q_{\text{accept}}, q_{\text{reject}}\right\}$
  - Non esiste nessuna configurazione C' tale che $C \to C'$
- **Di accettazione**: Se $C = uqv, u, v \in \Gamma^*$ e $q = q_{\text{accept}}$
- **Di rifiuto**: Se $C = uqv, u, v \in \Gamma^*$ e $q = q_{\text{reject}}$

7.6.1 Configurazioni Particolari

- Se $C = qv$ allora la testina è posizionata sulla prima cella del nastro

- Se $C = uq$ allora la testina è posizionata sulla prima cella della porzione del nastro contente solo $\sqcup$

7.7 Passo di Computazione

Sia $M = \big(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\big)$ una MdT deterministica
Siano $q_i, q_j \in Q,\ a, b, c \in \Gamma$, e $u, v \in \Gamma^*$

Diremo che

$$uaq_ibv \text{ produce } uq_jacv$$

Se $\delta(q_i, b) = \big(q_j, c, L\big)$ Diremo che

$$uaq_ibv \text{ produce } uacq_jv$$

Se $\delta(q_i, b) = \big(q_j, c, R\big)$

Quindi $C_1 \to C_2$ prende il nome di passo di computazione

7.8 Computazione

Siano C, C' configurazioni
$C \xrightarrow{*} C'$ se esistono configurazioni $C_1, ..., C_k, k \geq 1$ tali che:
- $C_1 = C$
- $C_i \to C_{i+1}$, per $i \in \{1, ..., k-1\}$
- $C_k = C'$

Diremo che $C \xrightarrow{*} C'$ è una **computazione**

Sia M una MdT e C una configurazione ci sono tre possibili casi:
- $C \xrightarrow{*} C'$ con $C' = uq_{\text{accept}}v$ configurazione di accettazione
  - M si ferma in q_{accept}
- $C \xrightarrow{*} C'$ con $C' = uq_{\text{reject}}v$ configurazione di rifiuto
  - M si ferma in q_{reject}
- Per ogni configurazione C' tale che $C \xrightarrow{*} C'$ esiste una configurazione C'' tale che $C \xrightarrow{*} C' \to C''$
  - M non si arresta

7.9 Parola Accetta da una MdT

Un MdT M accetta una parola $w \in \Sigma^*$ se esiste una computazione $C \xrightarrow{*} C'$, dove $C = q_0, w$ è la configurazione inziale di M con input w e $C' = uq_{\text{accept}}v$ è una configurazione di accettazione

Quindi M accetta $w \in \Sigma^*$ se e solo se esistono configurazioni $C_1, ..., C_k$ di M tali che:

- $C_1 = q_0 w$ è la configurazione inziale di M con input w
- $C_i \to C_{i+1}$ per ogni $i \in \{1, ..., k-1\}$
- C_k è una configurazione di accettazione

7.10 Parola Rifiutata da una MdT

Un MdT M rifiuta una parola $w \in \Sigma^*$ se esiste una computazione $C \xrightarrow{*} C'$, dove $C = q_0, w$ è la configurazione inziale di M con input w e $C' = uq_{\text{reject}}v$ è una configurazione di rifiuto

Quindi M si ferma su $w \in \Sigma^*$ se M accetta w oppure rifiuta w, cioè esistono configurazioni $C_1, ..., C_k$ di M tali che:

- $C_1 = q_0 w$ è la configurazione inziale di M con input w
- $C_i \to C_{i+1}$ per ogni $i \in \{1, ..., k-1\}$
- C_k è una configurazione di arresto

7.11 Linguaggio Riconosciuto da una MdT

Sia $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ un MdT, il linguaggio $L(M)$ riconosciuto da M è l'insieme delle stringhe che M accetta

$$L(M) = \left\{w \in \Sigma^* \mid \exists u, v \in \Gamma^* \; q_0 w \xrightarrow{*} u q_{\text{accept}} v\right\}$$

Quindi

$$L(M) = \{w \in \Sigma^* \mid M \text{ accetta } w\}$$

- Data una MdT M, esiste sempre il linguaggio riconosciuto da M ma non è detto che esista il linguaggio deciso da M, ciò è possibile se e solo se M è un decider
- Se M è un decider, esiste il linguaggio deciso da M e coincide con il linguaggio riconosciuto da M
- Se M non è un decider, esiste il linguaggio riconosciuto da M ma non il linguaggio deciso da M

7.12 MdT Decisore

Sia $R(M) = \{w \in \Sigma^* \mid M \text{ rifiuta } w\}$
In generale $L(M) \cup R(M)$ non coincide con Σ^*, ma se coincide allora M è un decider
Definizione Formale:
Un Mdt $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ è un decisore se per ogni $w \in \Sigma^*$, esistono $u, v \in \Gamma^*$ e $q \in \left\{q_{\text{accept}}, q_{\text{reject}}\right\}$ tali che:

$$q_0 w \xrightarrow{*} uqv$$

7.13 Linguaggio Deciso

Un linguaggio L è deciso da un MdT $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ se M è un decisore ed $L = L(M)$

Una MdT $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ decide un linguaggio L se per ogni $w \in \Sigma^*$, $q_0 w \xrightarrow{*} C$ con $C = uqv$ configurazione di arresto ed $L = L(M)$

7.14 Linguaggio Riconoscibile

Un linguaggio $L \subseteq \Sigma^*$ è Turing riconoscibile se esiste una macchina di Turing $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ tale che

- M riconosce L

$$L = L(M) = \left\{w \in \Sigma^* \mid \exists u, v \in \Gamma^* \ q_0 w \xrightarrow{*} u q_{\text{accept}} v\right\}$$

7.15 Linguaggio Decidibile

Un linguaggio $L \subseteq \Sigma^*$ è decidibile se esiste una macchina di Turing $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ tale che

- M riconosce L

$$L = L(M) = \left\{w \in \Sigma^* \mid \exists u, v \in \Gamma^* \ q_0 w \xrightarrow{*} u q_{\text{accept}} v\right\}$$

- M si arresta su ogni input

$$\text{Per ogni } w \in \Sigma^*, q_0 w \xrightarrow{*} uqv \text{ con } q \in \left\{q_{\text{accept}}, q_{\text{reject}}\right\}$$

Rappresenta un sottoinsieme proprio dei linguaggio Turing riconoscibili, quindi un linguaggio L è Turing riconoscibile ma non decidibile se:

- Esiste una MdT che riconosce L
- Non esiste nessuna MdT tale che M accetta tutte le stringhe di L e rifiuta tutte quelle che appartengono al complemento $\overline{L}$

7.16 Funzioni Calcolabili

Una funzione $f : \Sigma^* \to \Sigma^*$ è calcolabile se esiste una macchina di Turing $M = \left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}\right)$ tale che

$$\forall w \in \Sigma^* \ q_0 w \to q_{\text{accept}} f(w)$$

- La Mdt si deve arrestare su ogni input